Advanced Machine Learning

Lecture 6: FFT, im2col, & Winograd

Jens-Peter M. Zemke
zemke@tuhh.de

Institut für Mathematik
Lehrstuhl Numerische Mathematik
Technische Universität Hamburg

19 November 2024

Outline

Repetition
- History of CNN
- Toeplitz to circulant

Fast convolutions
- Separable kernels
- DFT to FFT
- im2col
- Winograd

Outline

Repetition
- History of CNN
- Toeplitz to circulant

Fast convolutions
- Separable kernels
- DFT to FFT
- im2col
- Winograd

The beginning

The first publication of a CNN was the “Neocognitron” by Fukushima [3] in 1980, where, based on biological findings about the cortex, fixed convolutions were used.

Backpropagation in a CNN was first used by LeCun et al. [8] in 1989. The nets that where constructed by LeCun and coworkers have the names LeNet-1 to LeNet-5 and differ in size and number of convolutions.

LeNet-5 by LeCun et al. [9] from 1998 was used on industrial scale for deciphering handwritten digits on checks and can be considered the first succesful CNN. The pooling used was some sort of averaging, activation function was tanh.

Max pooling was introduced by Yamaguchi et al. [12] for 1D in 1990 and by Weng, Ahuja, and Huang [10] for 2D in 1993.

The success of CNN started when they were trained on GPUs using CUDA instead of CPUs, see, e.g., Ciresan et al. [1], the most prominent CNN is AlexNet by Krizhevsky, Sutskever, and Hinton [5, 6]. More recent and much deeper is ResNet by He et al. [4].

AlexNet

AlexNet (2012) by Alex Krishevsky, Ilya Sutskever, Geoffrey E. Hinton, winner of ILSVRC (ImageNet Large Scale Visual Recognition Competition) Challenge in 2012

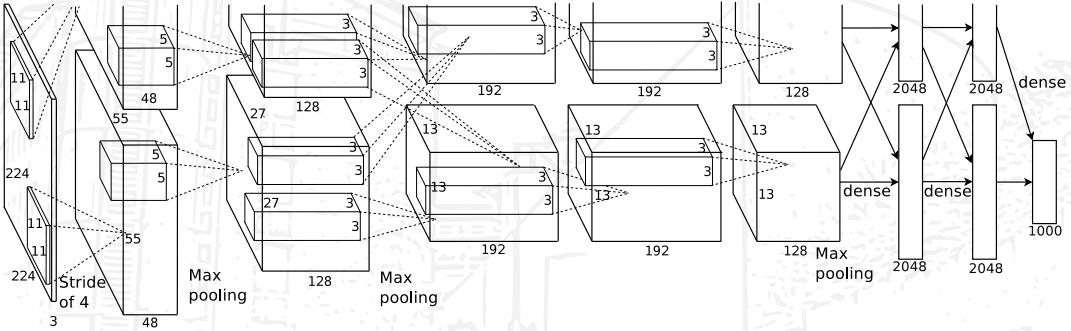
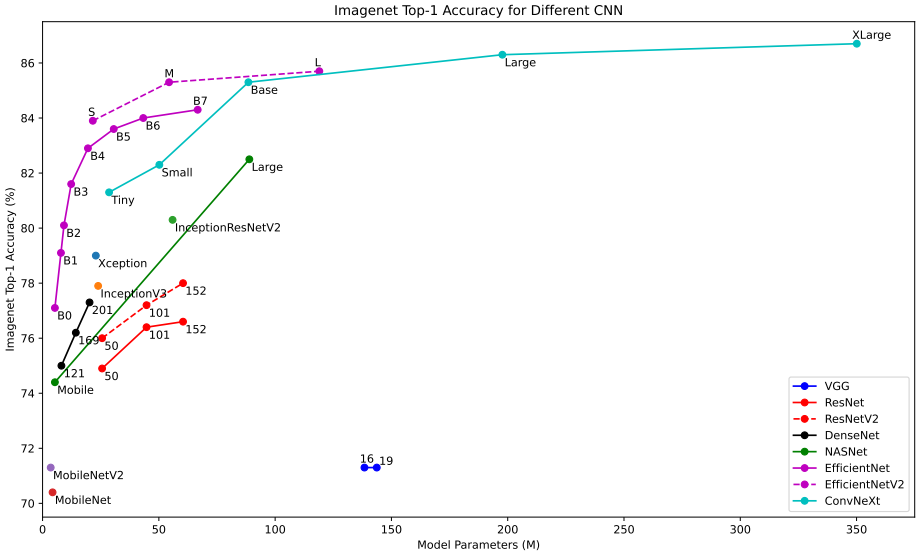


Image taken from Krizhevsky, Sutskever, and Hinton [5].

AlexNet has 8 layers, 5 convolutional followed by three dense layers. They introduced the ReLU activation function. Training on two GPUs for one week.

Overview of CNN (https://keras.io/api/applications/)



1D: Toeplitz to circulant

We rewrite the full convolution with the Toeplitz matrix $\mathbf{T}$ and obtain a square circulant matrix $\mathbf{C}$:

$$\begin{pmatrix} y_1 \\ \vdots \\ y_{k-1} \\ \hline y_k \\ \vdots \\ y_n \\ \hline y_{n+1} \\ \vdots \\ y_{n+k-1} \end{pmatrix} = \left(\begin{array}{ccccccc|ccc} f_1 & & & & & & & f_k & \cdots & f_2 \\ \vdots & \ddots & & & & & & & \ddots & \vdots \\ f_{k-1} & \cdots & f_1 & & & & & & & f_k \\ \hline f_k & \cdots & f_2 & f_1 & & & & & & \\ & \ddots & \ddots & \ddots & \ddots & & & & & \\ & & f_k & \cdots & f_2 & f_1 & & & & \\ \hline & & & f_k & \cdots & f_2 & f_1 & f_1 & & \\ & & & & \ddots & \vdots & & \vdots & \ddots & \\ & & & & & f_k & & f_{k-1} & \cdots & f_1 \end{array}\right) \begin{pmatrix} x_1 \\ \vdots \\ x_{k-1} \\ x_k \\ \vdots \\ x_n \\ \hline 0 \\ \vdots \\ 0 \end{pmatrix}, \quad \mathbf{C} \in \mathbb{R}^{(n+k-1)\times(n+k-1)}.$$

We write the above relation short-hand as $\mathbf{y} = \mathbf{C}\mathbf{x}$, where $\mathbf{x}$ has been extended by $k-1$ zeros.

1D: Fourier matrix and eigenvalues

The so-called Fourier matrix $\mathbf{F}$ is the complex symmetric matrix of (left and right) eigenvectors of any circulant matrix $\mathbf{C}$,

$$\mathbf{C} = \begin{pmatrix} c_0 & c_{n-1} & \cdots & c_1 \\ c_1 & c_0 & \ddots & \vdots \\ \vdots & \ddots & \ddots & c_{n-1} \\ c_{n-1} & \cdots & c_1 & c_0 \end{pmatrix}, \quad \mathbf{F} = \begin{pmatrix} \omega^{0\cdot 0} & \omega^{0\cdot 1} & \cdots & \omega^{0\cdot(n-1)} \\ \omega^{1\cdot 0} & \omega^{1\cdot 1} & \cdots & \omega^{1\cdot(n-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega^{(n-1)\cdot 0} & \omega^{(n-1)\cdot 1} & \cdots & \omega^{(n-1)\cdot(n-1)} \end{pmatrix}, \quad \omega = e^{-\frac{2\pi i}{n}}$$

This follows easily upon the observation that

$$\mathbf{C} = \sum_{j=0}^{n-1} \mathbf{P}^j c_j, \quad \mathbf{P} = \begin{pmatrix} 0 & \cdots & 0 & 1 \\ 1 & \ddots & \vdots & 0 \\ & \ddots & 0 & \vdots \\ & & 1 & 0 \end{pmatrix}, \quad \chi(z) = \det(z\mathbf{I} - \mathbf{P}) = z^n - 1.$$

1D: DFT & IDFT

Multiplication by the Fourier matrix is known as discrete Fourier transformation (DFT),

$$\mathbf{F}\mathbf{x} = \widehat{\mathbf{x}}, \quad \text{where} \quad \widehat{x}_k = \sum_{j=0}^{n-1} e^{-\frac{2\pi i k j}{n}} x_j = \sum_{j=0}^{n-1} \omega^{kj} x_j = \sum_{j=0}^{n-1} \left(\omega^k\right)^j x_j, \quad k = 0, \ldots, n-1.$$

The Fourier matrix has orthogonal columns of length n and is symmetric, so the inverse is

$$\mathbf{F}^{-1} = \frac{1}{n} \cdot \mathbf{F}^{\mathsf{H}} = \frac{1}{n} \cdot \overline{\mathbf{F}}$$

and the inverse DFT (IDFT) ist given by

$$\frac{1}{n}\overline{\mathbf{F}}\widehat{\mathbf{x}} = \mathbf{x}, \quad \text{where} \quad x_k = \frac{1}{n}\sum_{j=0}^{n-1} e^{\frac{2\pi i k j}{n}} \widehat{x}_j = \frac{1}{n}\sum_{j=0}^{n-1} \overline{\omega}^{kj} \widehat{x}_j = \frac{1}{n}\sum_{j=0}^{n-1} \left(\overline{\omega}^k\right)^j \widehat{x}_j, \quad k = 0, \ldots, n-1.$$

Convolution Theorem (1D & 2D)

Theorem

The DFT maps convolution to elementwise multiplication.

1D: expand input $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{f} \in \mathbb{R}^k$ by zeros to obtain vectors $\widetilde{\mathbf{x}}, \widetilde{\mathbf{f}} \in \mathbb{R}^{n+k-1}$, $\mathbf{y}$ is the full convolution,

$$\mathbf{y} = \mathbf{f} * \mathbf{x} \quad \Leftrightarrow \quad \widehat{\mathbf{y}} = \widehat{\mathbf{f}} \circ \widehat{\mathbf{x}} \quad \Leftrightarrow \quad \mathbf{y} = \mathrm{IDFT}(\mathrm{DFT}(\widetilde{\mathbf{f}}) \circ \mathrm{DFT}(\widetilde{\mathbf{x}})).$$

2D: append zeros to rows/columns and use the DFT/IDFT such that

$$\mathbf{Y} = \mathbf{F} * \mathbf{X} \quad \Leftrightarrow \quad \widehat{\mathbf{Y}} = \widehat{\mathbf{F}} \circ \widehat{\mathbf{X}} \quad \Leftrightarrow \quad \mathbf{Y} = \mathrm{IDFT}(\mathrm{DFT}(\widetilde{\mathbf{F}}) \circ \mathrm{DFT}(\widetilde{\mathbf{X}})).$$

2D: convolution to block circulant w/ circulant blocks

"valid" convolution $\rightsquigarrow$ full convolution $\rightsquigarrow$ circulant blocks $\rightsquigarrow$ block full convolution $\rightsquigarrow$ block circulant

$$
\begin{pmatrix} y_{0,0} \\ y_{0,1} \\ y_{0,2} \\ y_{0,3} \\ y_{1,0} \\ y_{1,1} \\ y_{1,2} \\ y_{1,3} \\ y_{2,0} \\ y_{2,1} \\ y_{2,2} \\ y_{2,3} \\ y_{3,0} \\ y_{3,1} \\ y_{3,2} \\ y_{3,3} \end{pmatrix} =
\left(\begin{array}{cccc|cccc|cccc|cccc}
f_{1,1} & 0 & 0 & f_{1,2} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,1} & 0 & 0 & f_{2,2} \\
f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 \\
0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 \\
0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} \\
\hline
f_{2,1} & 0 & 0 & f_{2,2} & f_{1,1} & 0 & 0 & f_{1,2} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
\hline
0 & 0 & 0 & 0 & f_{2,1} & 0 & 0 & f_{2,2} & f_{1,1} & 0 & 0 & f_{1,2} & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 \\
\hline
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,1} & 0 & 0 & f_{2,2} & f_{1,1} & 0 & 0 & f_{1,2} \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1}
\end{array}\right)
\begin{pmatrix} x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ 0 \\ x_{2,1} \\ x_{2,2} \\ x_{2,3} \\ 0 \\ x_{3,1} \\ x_{3,2} \\ x_{3,3} \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}.
$$

2D: DFT & IDFT

In matrix form we obtain:

- extending the input from a matrix $\mathbf{X} \in \mathbb{R}^{n_1 \times n_2}$ by zero-padding to $\widetilde{\mathbf{X}} \in \mathbb{R}^{(n_1+k_1-1)\times(n_2+k_2-1)}$,

$$\mathbf{X} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{pmatrix} \quad \rightsquigarrow \quad \widetilde{\mathbf{X}} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} & 0 \\ x_{2,1} & x_{2,2} & x_{2,3} & 0 \\ x_{3,1} & x_{3,2} & x_{3,3} & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix},$$

  similarly for the kernel $\mathbf{K} \in \mathbb{R}^{k_1 \times k_2}$ to $\widetilde{\mathbf{K}} \in \mathbb{R}^{(n_1+k_1-1)\times(n_2+k_2-1)}$,

- 2D DFT of the extended input and kernel, componentwise multiplication,

$$\widehat{\mathbf{X}} = \mathbf{F}_{n_1+k_1-1}\widetilde{\mathbf{X}}\mathbf{F}_{n_2+k_2-1}, \quad \widehat{\mathbf{K}} = \mathbf{F}_{n_1+k_1-1}\widetilde{\mathbf{K}}\mathbf{F}_{n_2+k_2-1}, \quad \widehat{\mathbf{Y}} = \widehat{\mathbf{K}} \circ \widehat{\mathbf{X}},$$

- 2D IDFT and selection of region of interest to obtain the convolution,

$$\widetilde{\mathbf{Y}} = \frac{1}{(n_1+k_1-1)(n_2+k_2-1)}\overline{\mathbf{F}}\widehat{\mathbf{Y}}\overline{\mathbf{F}}, \quad \mathbf{Y} = \widetilde{\mathbf{Y}}[k_1-1:n_1, k_2-1:n_2].$$

Outline

Repetition
History of CNN
Toeplitz to circulant

Fast convolutions
Separable kernels
DFT to FFT
im2col
Winograd

2D: separable kernels

The 2D DFT consists of two 1D DFT, one applied to all column vectors, one applied to all row vectors. The order does not matter.

2D: separable kernels

The 2D DFT consists of two 1D DFT, one applied to all column vectors, one applied to all row vectors. The order does not matter.

If, e.g., the kernel is a rank-one matrix $\mathbf{K} = \mathbf{k}_1 \mathbf{k}_2^\mathsf{T}$, then we can group both DFT to obtain the rank-one 2D DFT

$$\widehat{\mathbf{K}} = (\mathbf{F}\mathbf{k}_1)(\mathbf{F}\mathbf{k}_2)^\mathsf{T}.$$

2D: separable kernels

The 2D DFT consists of two 1D DFT, one applied to all column vectors, one applied to all row vectors. The order does not matter.

If, e.g., the kernel is a rank-one matrix $\mathbf{K} = \mathbf{k}_1 \mathbf{k}_2^\mathsf{T}$, then we can group both DFT to obtain the rank-one 2D DFT

$$\widehat{\mathbf{K}} = (\mathbf{F}\mathbf{k}_1)(\mathbf{F}\mathbf{k}_2)^\mathsf{T}.$$

Separable kernels are cheaper to apply. Examples include the Sobel kernels and the simple smoothing kernels,

$$\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} \begin{pmatrix} -1 & 0 & 1 \end{pmatrix}, \quad \begin{pmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 1 \end{pmatrix}.$$

dD: DFT & IDFT

The DFT and IDFT extend naturally to the dD case for any $d \in \mathbb{N}$. We define the nth root of unity

$$\omega_n := e^{-\frac{2\pi i}{n}}.$$

dD: DFT & IDFT

The DFT and IDFT extend naturally to the dD case for any $d \in \mathbb{N}$. We define the nth root of unity

$$\omega_n := e^{-\frac{2\pi i}{n}}.$$

Written in components, the 1D DFT of $\mathbf{x} \in \mathbb{R}^n$ and the 2D DFT of $\mathbf{X} \in \mathbb{R}^{n_1 \times n_2}$ are given by

$$\widehat{x}_k = \sum_{j=0}^{n-1} (\omega_n^k)^j x_j, \qquad \widehat{x}_{k_1,k_2} = \sum_{j_1=0}^{n_1-1} \sum_{j_2=0}^{n_2-1} (\omega_{n_1}^{k_1})^{j_1} (\omega_{n_2}^{k_2})^{j_2} x_{j_1,j_2},$$

dD: DFT & IDFT

The DFT and IDFT extend naturally to the dD case for any $d \in \mathbb{N}$. We define the nth root of unity

$$\omega_n := e^{-\frac{2\pi i}{n}}.$$

Written in components, the 1D DFT of $\mathbf{x} \in \mathbb{R}^n$ and the 2D DFT of $\mathbf{X} \in \mathbb{R}^{n_1 \times n_2}$ are given by

$$\widehat{x}_k = \sum_{j=0}^{n-1} (\omega_n^k)^j x_j, \qquad \widehat{x}_{k_1,k_2} = \sum_{j_1=0}^{n_1-1} \sum_{j_2=0}^{n_2-1} (\omega_{n_1}^{k_1})^{j_1} (\omega_{n_2}^{k_2})^{j_2} x_{j_1,j_2},$$

the dD DFT (similarly for the dD IDFT) of $\mathbf{X} \in \mathbb{R}^{\mathbf{n}}$, $\mathbf{n} \in \mathbb{N}^d$ is defined by

$$\widehat{x}_{\mathbf{k}} := \sum_{\mathbf{j}=\mathbf{o}}^{\mathbf{n}-\mathbf{e}} (\omega_{\mathbf{n}}^{\mathbf{k}})^{\mathbf{j}} x_{\mathbf{j}} := \sum_{j_1=0}^{n_1-1} \cdots \sum_{j_d=0}^{n_d-1} (\omega_{n_1}^{k_1})^{j_1} \cdots (\omega_{n_d}^{k_d})^{j_d} x_{j_1,\ldots,j_d}.$$

Why should we use the DFT?

The DFT and IDFT correspond to multiplications with matrices in $\mathbb{R}^{(n+k-1)^d}$.

Why should we use the DFT?

The DFT and IDFT correspond to multiplications with matrices in $\mathbb{R}^{(n+k-1)^d}$. Why should we

- enlarge our problem,
- increase the number of matrix multiplications, and
- use complex arithmetic for real values?

Why should we use the DFT?

The DFT and IDFT correspond to multiplications with matrices in $\mathbb{R}^{(n+k-1)^d}$. Why should we

- enlarge our problem,
- increase the number of matrix multiplications, and
- use complex arithmetic for real values?

The answer is the so-called fast Fourier transform (FFT) by Cooley and Tukey [2]. Using the FFT, the DFT of size n costs $O(n \log(n))$ operations compared to $O(n^2)$ when implemented naively.

Why should we use the DFT?

The DFT and IDFT correspond to multiplications with matrices in $\mathbb{R}^{(n+k-1)^d}$. Why should we

- enlarge our problem,
- increase the number of matrix multiplications, and
- use complex arithmetic for real values?

The answer is the so-called fast Fourier transform (FFT) by Cooley and Tukey [2]. Using the FFT, the DFT of size n costs $O(n \log(n))$ operations compared to $O(n^2)$ when implemented naively.

There are several variants of the FFT. Software libraries like FFTW (the fastest Fourier transform in the west) try to find the best FFT for a given task and generate code, which is then executed.

Why should we use the DFT?

The DFT and IDFT correspond to multiplications with matrices in $\mathbb{R}^{(n+k-1)^d}$. Why should we

- enlarge our problem,
- increase the number of matrix multiplications, and
- use complex arithmetic for real values?

The answer is the so-called fast Fourier transform (FFT) by Cooley and Tukey [2]. Using the FFT, the DFT of size n costs $O(n \log(n))$ operations compared to $O(n^2)$ when implemented naively.

There are several variants of the FFT. Software libraries like FFTW (the fastest Fourier transform in the west) try to find the best FFT for a given task and generate code, which is then executed.

FFT based convolution pays off when the kernel is quite large, for smaller kernels there are other approaches. SciPy's `convolve` uses the FFT when k is $O(\log(n))$ or larger.

1D FFT

We derive the FFT in the 1D case. The 1D DFT is the task to compute

$$\widehat{x}_k = \sum_{j=0}^{n-1} (\omega_n^k)^j x_j, \quad k = 0, \ldots, n-1, \quad \omega_n = e^{-\frac{2\pi i}{n}}.$$

1D FFT

We derive the FFT in the 1D case. The 1D DFT is the task to compute

$$\widehat{x}_k = \sum_{j=0}^{n-1} (\omega_n^k)^j x_j, \quad k = 0, \ldots, n-1, \quad \omega_n = e^{-\frac{2\pi i}{n}}.$$

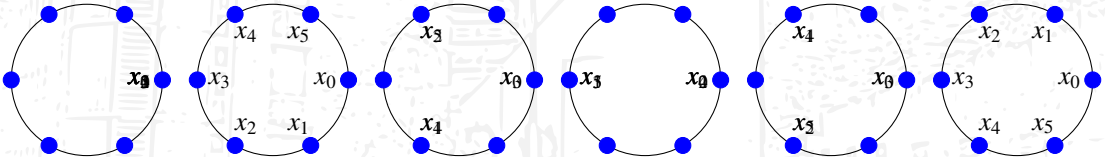
Suppose that $n = n_1 \cdot n_2$ is compound, for example $6 = 2 \cdot 3$.

1D FFT

We derive the FFT in the 1D case. The 1D DFT is the task to compute

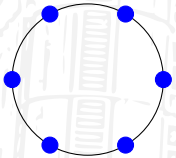
$$\widehat{x}_k = \sum_{j=0}^{n-1} (\omega_n^k)^j x_j, \quad k = 0, \ldots, n-1, \quad \omega_n = e^{-\frac{2\pi i}{n}}.$$

Suppose that $n = n_1 \cdot n_2$ is compound, for example $6 = 2 \cdot 3$. Then we want to compute the following $n = 6$ sums of complex values, which are linear combinations of sixth roots of unity:



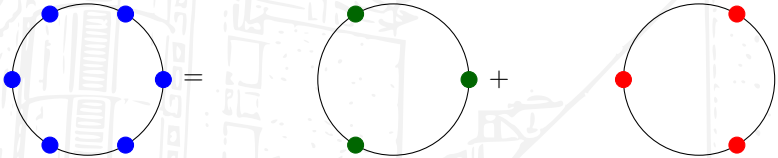
1D FFT: compound n

Now comes the crucial point: since $6 = 2 \cdot 3$ is compound,



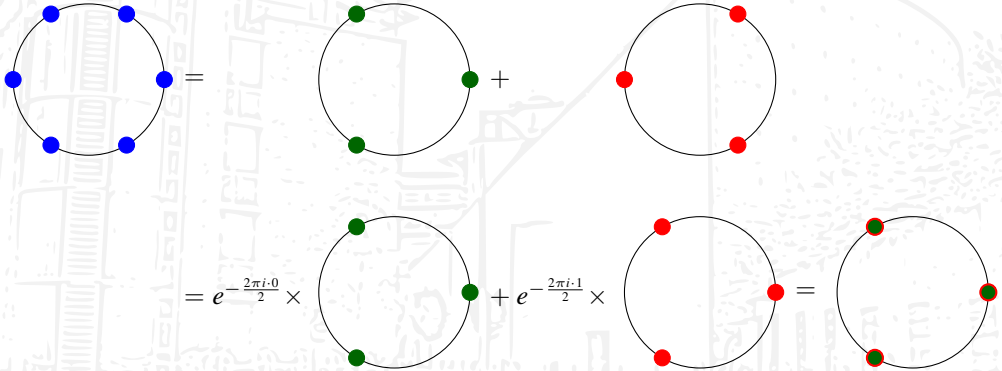
1D FFT: compound n

Now comes the crucial point: since $6 = 2 \cdot 3$ is compound, we can find 2 occurrences of the third roots of unity,



1D FFT: compound n

Now comes the crucial point: since $6 = 2 \cdot 3$ is compound, we can find 2 occurrences of the third roots of unity, flipped by the second roots of unity,



1D FFT: decomposition of the Fourier matrix

Inserting the coefficients gives relations involving powers ω_6^k for $k = 0, 1, 2,$

1D FFT: decomposition of the Fourier matrix

Inserting the coefficients gives relations involving powers ω_6^k for $k = 0, 1, 2$, in general, for $n = 2m$:

$$\widehat{x}_k = \sum_{j=0}^{2m-1} (\omega_{2m}^k)^j x_j$$

1D FFT: decomposition of the Fourier matrix

Inserting the coefficients gives relations involving powers ω_6^k for $k = 0, 1, 2$, in general, for $n = 2m$:

$$\widehat{x}_k = \sum_{j=0}^{2m-1} (\omega_{2m}^k)^j x_j, \qquad k = mp + q, \quad j = 2r + \ell, \quad p, \ell \in \{0, 1\} : \quad (\omega_{2m}^k)^j = \omega_{2m}^{(mp+q)(2r+\ell)}$$

1D FFT: decomposition of the Fourier matrix

Inserting the coefficients gives relations involving powers ω_6^k for $k = 0, 1, 2$, in general, for $n = 2m$:

$$\widehat{x}_k = \sum_{j=0}^{2m-1} (\omega_{2m}^k)^j x_j, \qquad k = mp + q, \quad j = 2r + \ell, \quad p, \ell \in \{0, 1\} : \quad (\omega_{2m}^k)^j = \omega_{2m}^{2mpr + mp\ell + 2qr + q\ell}$$

1D FFT: decomposition of the Fourier matrix

Inserting the coefficients gives relations involving powers ω_6^k for $k = 0, 1, 2$, in general, for $n = 2m$:

$$\widehat{x}_k = \sum_{j=0}^{2m-1} (\omega_{2m}^k)^j x_j, \qquad k = mp + q, \quad j = 2r + \ell, \quad p, \ell \in \{0, 1\} : \quad (\omega_{2m}^k)^j = \omega_2^{p\ell} \omega_m^{qr} \omega_n^{q\ell}$$

1D FFT: decomposition of the Fourier matrix

Inserting the coefficients gives relations involving powers ω_6^k for $k = 0, 1, 2$, in general, for $n = 2m$:

$$\widehat{x}_k = \sum_{j=0}^{2m-1} (\omega_{2m}^k)^j x_j = \sum_{r=0}^{m-1} (\omega_m^q)^r x_{2r} + \omega_2^p \omega_n^q \sum_{r=0}^{m-1} (\omega_m^q)^r x_{2r+1}, \qquad (\omega_{2m}^k)^j = \omega_2^{p\ell} \omega_m^{qr} \omega_n^{q\ell}$$

1D FFT: decomposition of the Fourier matrix

Inserting the coefficients gives relations involving powers ω_6^k for $k = 0, 1, 2$, in general, for $n = 2m$:

$$\widehat{x}_k = \sum_{j=0}^{2m-1} (\omega_{2m}^k)^j x_j = \begin{cases} \sum_{r=0}^{m-1} (\omega_m^q)^r x_{2r} + \omega_n^q \sum_{r=0}^{m-1} (\omega_m^q)^r x_{2r+1}, & q = 0, \ldots, m-1, \quad k = 0 \cdot m + q, \\ \end{cases}$$

1D FFT: decomposition of the Fourier matrix

Inserting the coefficients gives relations involving powers ω_6^k for $k = 0, 1, 2$, in general, for $n = 2m$:

$$\widehat{x}_k = \sum_{j=0}^{2m-1} (\omega_{2m}^k)^j x_j = \begin{cases} \sum_{r=0}^{m-1} (\omega_m^q)^r x_{2r} + \omega_n^q \sum_{r=0}^{m-1} (\omega_m^q)^r x_{2r+1}, & q = 0, \ldots, m-1, \quad k = 0 \cdot m + q, \\ \sum_{r=0}^{m-1} (\omega_m^q)^r x_{2r} - \omega_n^q \sum_{r=0}^{m-1} (\omega_m^q)^r x_{2r+1}, & q = 0, \ldots, m-1, \quad k = 1 \cdot m + q. \end{cases}$$

1D FFT: decomposition of the Fourier matrix

Inserting the coefficients gives relations involving powers ω_6^k for $k = 0, 1, 2$, in general, for $n = 2m$:

$$\widehat{x}_k = \sum_{j=0}^{2m-1} (\omega_{2m}^k)^j x_j = \begin{cases} \sum_{r=0}^{m-1} (\omega_m^q)^r x_{2r} + \omega_n^q \sum_{r=0}^{m-1} (\omega_m^q)^r x_{2r+1}, & q = 0, \ldots, m-1, \quad k = 0 \cdot m + q, \\ \sum_{r=0}^{m-1} (\omega_m^q)^r x_{2r} - \omega_n^q \sum_{r=0}^{m-1} (\omega_m^q)^r x_{2r+1}, & q = 0, \ldots, m-1, \quad k = 1 \cdot m + q. \end{cases}$$

This DFT is given by two smaller DFTs, the Fourier matrix $\mathbf{F}_6$ of the previous example can be decomposed as

$$\mathbf{F}_6(:, [0,2,4,1,3,5]) = \left(\begin{array}{ccc|ccc} 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & \omega_6 & 0 \\ 0 & 0 & 1 & 0 & 0 & \omega_6^2 \\ \hline 1 & 0 & 0 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 & -\omega_6 & 0 \\ 0 & 0 & 1 & 0 & 0 & -\omega_6^2 \end{array}\right) \begin{pmatrix} \mathbf{F}_3 & \mathbf{O} \\ \mathbf{O} & \mathbf{F}_3 \end{pmatrix}, \quad \begin{pmatrix} \mathbf{F}_3 & \mathbf{O} \\ \mathbf{O} & \mathbf{F}_3 \end{pmatrix} = \mathbf{I}_2 \otimes \mathbf{F}_3.$$

1D FFT: radix-2 decimation in time (DIT) FFT

For highly compound n, e.g., $n = 2^k$, the FFT can be computed recursively in $O(kn) = O(n \log(n))$:

```
1   def ditfft(x):
2     n = len(x)
3     if n == 1:
4         y = x
5     else:
6         y = np.zeros(n, dtype='complex')
7         m = n // 2
8         omega = np.exp(-2 * np.pi * 1j / n)
9         d = omega ** np.arange(m)
10        z_top = ditfft(x[0:n:2])
11        z_bot = ditfft(x[1:n:2])
12        y[0:m] = z_top + d * z_bot
13        y[m:n] = z_top - d * z_bot
14    return y
```

1D FFT: radix-2 decimation in time (DIT) FFT

For highly compound n, e.g., $n = 2^k$, the FFT can be computed recursively in $O(kn) = O(n \log(n))$:

```
1   def ditfft(x):
2     n = len(x)
3     if n == 1:
4         y = x
5     else:
6         y = np.zeros(n, dtype='complex')
7         m = n // 2
8         omega = np.exp(-2 * np.pi * 1j / n)
9         d = omega ** np.arange(m)
10        z_top = ditfft(x[0:n:2])
11        z_bot = ditfft(x[1:n:2])
12        y[0:m] = z_top + d * z_bot
13        y[m:n] = z_top - d * z_bot
14    return y
```

To implement a non-recursive variant we need to understand the permutation of the indices.

1D FFT: radix-2 DIT FFT index movements

Suppose that $n = 2^4 = 16$, $k = 4$. Then by repeated grouping of the indices we obtain the orders

$k = 0$ $\quad [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]$

1D FFT: radix-2 DIT FFT index movements

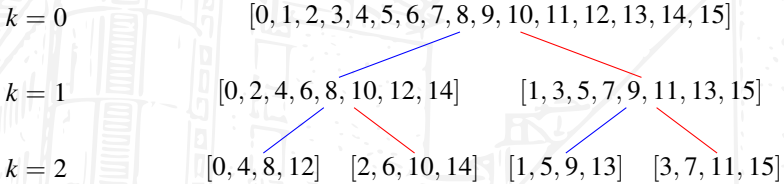
Suppose that $n = 2^4 = 16$, $k = 4$. Then by repeated grouping of the indices we obtain the orders

$k = 0$ $\quad [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]$

$k = 1$ $\quad [0, 2, 4, 6, 8, 10, 12, 14] \quad [1, 3, 5, 7, 9, 11, 13, 15]$

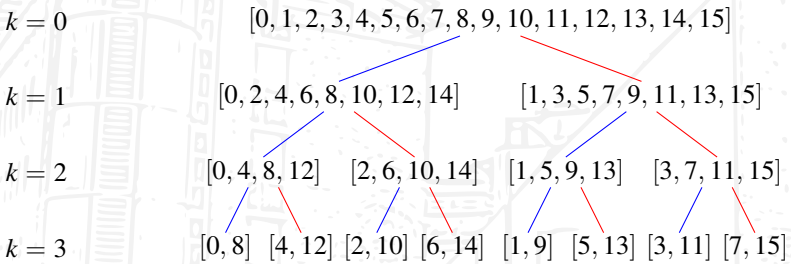
1D FFT: radix-2 DIT FFT index movements

Suppose that $n = 2^4 = 16$, $k = 4$. Then by repeated grouping of the indices we obtain the orders



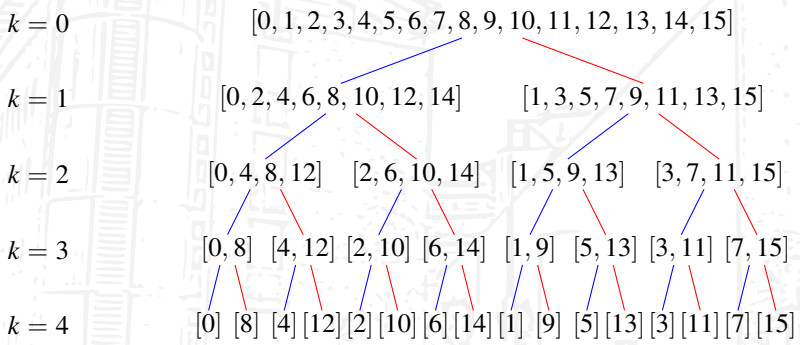
1D FFT: radix-2 DIT FFT index movements

Suppose that $n = 2^4 = 16$, $k = 4$. Then by repeated grouping of the indices we obtain the orders



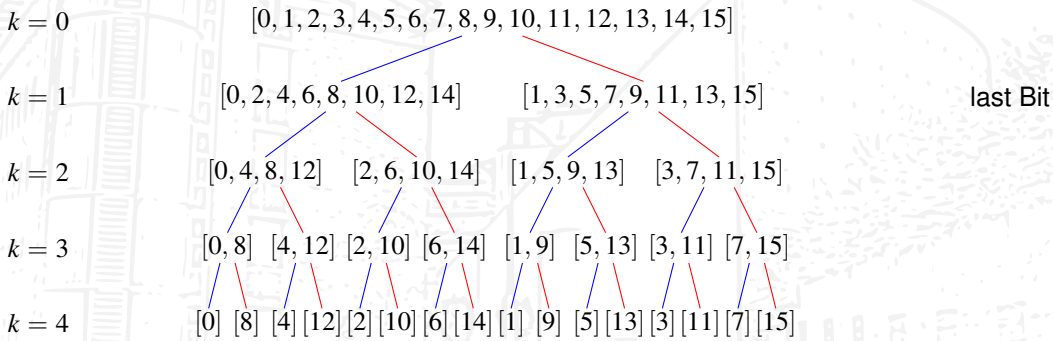
1D FFT: radix-2 DIT FFT index movements

Suppose that $n = 2^4 = 16$, $k = 4$. Then by repeated grouping of the indices we obtain the orders



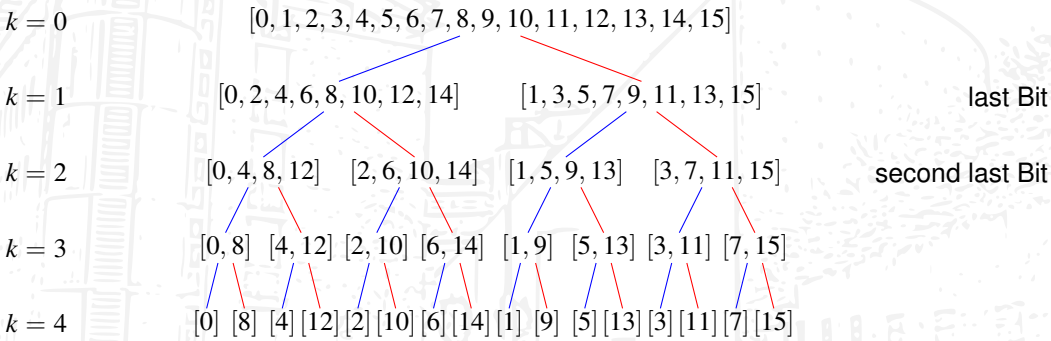
1D FFT: radix-2 DIT FFT index movements

Suppose that $n = 2^4 = 16$, $k = 4$. Then by repeated grouping of the indices we obtain the orders



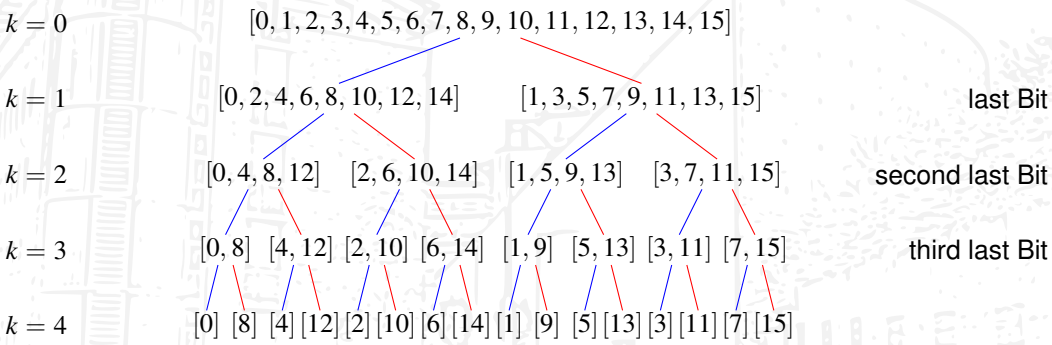
1D FFT: radix-2 DIT FFT index movements

Suppose that $n = 2^4 = 16$, $k = 4$. Then by repeated grouping of the indices we obtain the orders



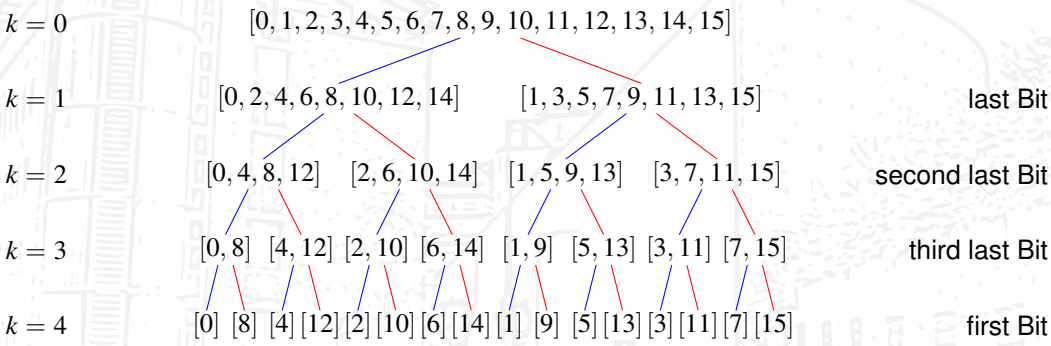
1D FFT: radix-2 DIT FFT index movements

Suppose that $n = 2^4 = 16$, $k = 4$. Then by repeated grouping of the indices we obtain the orders



1D FFT: radix-2 DIT FFT index movements

Suppose that $n = 2^4 = 16$, $k = 4$. Then by repeated grouping of the indices we obtain the orders



1D FFT: radix-2 DIT FFT index movements

The permutation $\mathbf{P}_{16}$ that computes the order of the elements corresponds to reverting the bits.

$$\begin{pmatrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \\ 10 \\ 11 \\ 12 \\ 13 \\ 14 \\ 15 \end{pmatrix} = \begin{pmatrix} 0000_2 \\ 0001_2 \\ 0010_2 \\ 0011_2 \\ 0100_2 \\ 0101_2 \\ 0110_2 \\ 0111_2 \\ 1000_2 \\ 1001_2 \\ 1010_2 \\ 1011_2 \\ 1100_2 \\ 1101_2 \\ 1110_2 \\ 1111_2 \end{pmatrix} \rightsquigarrow \begin{pmatrix} 0000_2 \\ 1000_2 \\ 0100_2 \\ 1100_2 \\ 0010_2 \\ 1010_2 \\ 0110_2 \\ 1110_2 \\ 0001_2 \\ 1001_2 \\ 0101_2 \\ 1101_2 \\ 0011_2 \\ 1011_2 \\ 0111_2 \\ 1111_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 8 \\ 4 \\ 12 \\ 2 \\ 10 \\ 6 \\ 14 \\ 1 \\ 9 \\ 5 \\ 13 \\ 3 \\ 11 \\ 7 \\ 15 \end{pmatrix}$$

1D FFT: radix-2 DIT FFT index movements

The permutation $\mathbf{P}_{16}$ that computes the order of the elements corresponds to reverting the bits.

Remember that $\omega_\ell = e^{-\frac{2\pi i}{\ell}}$.

$$\begin{pmatrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \\ 10 \\ 11 \\ 12 \\ 13 \\ 14 \\ 15 \end{pmatrix} = \begin{pmatrix} 0000_2 \\ 0001_2 \\ 0010_2 \\ 0011_2 \\ 0100_2 \\ 0101_2 \\ 0110_2 \\ 0111_2 \\ 1000_2 \\ 1001_2 \\ 1010_2 \\ 1011_2 \\ 1100_2 \\ 1101_2 \\ 1110_2 \\ 1111_2 \end{pmatrix} \rightsquigarrow \begin{pmatrix} 0000_2 \\ 1000_2 \\ 0100_2 \\ 1100_2 \\ 0010_2 \\ 1010_2 \\ 0110_2 \\ 1110_2 \\ 0001_2 \\ 1001_2 \\ 0101_2 \\ 1101_2 \\ 0011_2 \\ 1011_2 \\ 0111_2 \\ 1111_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 8 \\ 4 \\ 12 \\ 2 \\ 10 \\ 6 \\ 14 \\ 1 \\ 9 \\ 5 \\ 13 \\ 3 \\ 11 \\ 7 \\ 15 \end{pmatrix}$$

1D FFT: radix-2 DIT FFT index movements

The permutation $\mathbf{P}_{16}$ that computes the order of the elements corresponds to reverting the bits.

Remember that $\omega_\ell = e^{-\frac{2\pi i}{\ell}}$.We obtain the so-called Cooley-Tukey Factorization

$$\mathbf{F}_{2^k} = \mathbf{B}_k \mathbf{B}_{k-1} \cdots \mathbf{B}_1 \mathbf{P}_{2^k}$$

$$\begin{pmatrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \\ 10 \\ 11 \\ 12 \\ 13 \\ 14 \\ 15 \end{pmatrix} = \begin{pmatrix} 0000_2 \\ 0001_2 \\ 0010_2 \\ 0011_2 \\ 0100_2 \\ 0101_2 \\ 0110_2 \\ 0111_2 \\ 1000_2 \\ 1001_2 \\ 1010_2 \\ 1011_2 \\ 1100_2 \\ 1101_2 \\ 1110_2 \\ 1111_2 \end{pmatrix} \rightsquigarrow \begin{pmatrix} 0000_2 \\ 1000_2 \\ 0100_2 \\ 1100_2 \\ 0010_2 \\ 1010_2 \\ 0110_2 \\ 1110_2 \\ 0001_2 \\ 1001_2 \\ 0101_2 \\ 1101_2 \\ 0011_2 \\ 1011_2 \\ 0111_2 \\ 1111_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 8 \\ 4 \\ 12 \\ 2 \\ 10 \\ 6 \\ 14 \\ 1 \\ 9 \\ 5 \\ 13 \\ 3 \\ 11 \\ 7 \\ 15 \end{pmatrix}$$

1D FFT: radix-2 DIT FFT index movements

The permutation $\mathbf{P}_{16}$ that computes the order of the elements corresponds to reverting the bits.

Remember that $\omega_\ell = e^{-\frac{2\pi i}{\ell}}$.We obtain the so-called Cooley-Tukey Factorization

$$\mathbf{F}_{2^k} = \mathbf{B}_k \mathbf{B}_{k-1} \cdots \mathbf{B}_1 \mathbf{P}_{2^k}$$

with the butterfly matrices

$$\mathbf{B}_j := \mathbf{I}_{2^{k-j}} \otimes \begin{pmatrix} \mathbf{I}_{2^{j-1}} & \mathbf{\Omega}_{2^{j-1}} \\ \mathbf{I}_{2^{j-1}} & -\mathbf{\Omega}_{2^{j-1}} \end{pmatrix},$$

where

$$\mathbf{\Omega}_{2^{j-1}} := \mathrm{diag}(1, \omega_{2^j}, \ldots, \omega_{2^j}^{2^{j-1}-1}).$$

$$\begin{pmatrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \\ 10 \\ 11 \\ 12 \\ 13 \\ 14 \\ 15 \end{pmatrix} = \begin{pmatrix} 0000_2 \\ 0001_2 \\ 0010_2 \\ 0011_2 \\ 0100_2 \\ 0101_2 \\ 0110_2 \\ 0111_2 \\ 1000_2 \\ 1001_2 \\ 1010_2 \\ 1011_2 \\ 1100_2 \\ 1101_2 \\ 1110_2 \\ 1111_2 \end{pmatrix} \rightsquigarrow \begin{pmatrix} 0000_2 \\ 1000_2 \\ 0100_2 \\ 1100_2 \\ 0010_2 \\ 1010_2 \\ 0110_2 \\ 1110_2 \\ 0001_2 \\ 1001_2 \\ 0101_2 \\ 1101_2 \\ 0011_2 \\ 1011_2 \\ 0111_2 \\ 1111_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 8 \\ 4 \\ 12 \\ 2 \\ 10 \\ 6 \\ 14 \\ 1 \\ 9 \\ 5 \\ 13 \\ 3 \\ 11 \\ 7 \\ 15 \end{pmatrix}$$

1D FFT: Gentleman-Sande & convolutions

Gentleman-Sande: $\mathbf{F}_n = \mathbf{F}_n^\mathsf{T}$, thus

$$\mathbf{F}_{2^k} = \mathbf{P}_{2^k}^\mathsf{T} \mathbf{B}_1^\mathsf{T} \cdots \mathbf{B}_{k-1}^\mathsf{T} \mathbf{B}_k^\mathsf{T}.$$

FFT and IFFT (w/o normalizing factor) given by

```
1    for j in reversed(range(k)):
2        x = Bj.T @ x
3    x = Pn.T @ x
```

```
1    x = Pn @ x
2    for j in range(k):
3        x = Bj.conj() @ x
```

1D FFT: Gentleman-Sande & convolutions

Gentleman-Sande: $\mathbf{F}_n = \mathbf{F}_n^\mathsf{T}$, thus

$$\mathbf{F}_{2^k} = \mathbf{P}_{2^k}^\mathsf{T} \mathbf{B}_1^\mathsf{T} \cdots \mathbf{B}_{k-1}^\mathsf{T} \mathbf{B}_k^\mathsf{T}.$$

FFT and IFFT (w/o normalizing factor) given by

```
1    for j in reversed(range(k)):
2        x = Bj.T @ x
3    x = Pn.T @ x
```

```
1    x = Pn @ x
2    for j in range(k):
3        x = Bj.conj() @ x
```

Convolution: omit $\mathbf{x} \leftarrow \mathbf{P}_n^\mathsf{T}\mathbf{x}$ (FFT) and $\mathbf{x} \leftarrow \mathbf{P}_n\mathbf{x}$ (IFFT) and work in the scrambled DFT space.

2D FFT

The 2D DFT is given by

$$\mathbf{F}_n \mathbf{X} \mathbf{F}_n, \quad \mathbf{F}_n = \mathbf{A}_1 \mathbf{A}_2 \cdots \mathbf{A}_k.$$

Could be implemented as one rowwise FFT and one columnwise FFT in any order.

To ensure data locality better use update

```
for j in reversed(range(k)):
    X = Aj @ X @ Aj.T
```

intermingling both FFTs often is faster.

Python, convolutions, and the FFT

Python offers different implementations of convolution / FFT.

Python, convolutions, and the FFT

Python offers different implementations of convolution / FFT.

SciPy:

- `scipy.signal.convolve`
- `scipy.signal.convolve2d`
- `scipy.signal.fftconvolve`
- `scipy.signal.oaconvolve`
- `scipy.ndimage.convolve`

Python, convolutions, and the FFT

Python offers different implementations of convolution / FFT.

SciPy:

- `scipy.signal.convolve`
- `scipy.signal.convolve2d`
- `scipy.signal.fftconvolve`
- `scipy.signal.oaconvolve`
- `scipy.ndimage.convolve`

Numpy: `numpy.fft` (`r`: real input)

- `fft`, `ifft`, `rfft`, `irfft`: 1D
- `fft2`, `ifft2`, `rfft2`, `irfft2`: 2D (input $\geqslant$2D!)
- `fftn`, `ifftn`, `rfftn`, `irfftn`: nD

Convolution based on BLAS (GEMM)

The matrix-matrix multiplication in the BLAS Level 3 (`GEMM` = GEneral matrix Matrix-Matrix multiplication) is fast.

Convolution based on BLAS (GEMM)

The matrix-matrix multiplication in the BLAS Level 3 (`GEMM` = GEneral matrix Matrix-Matrix multiplication) is fast.

Can we use `GEMM` for convolution?

Convolution based on BLAS (GEMM)

The matrix-matrix multiplication in the BLAS Level 3 (`GEMM` = GEneral matrix Matrix-Matrix multiplication) is fast.

Can we use `GEMM` for convolution? The task is to compute the sum over all channels c of a 2D image of size $h \times w$ for a minibatch n of images, here

$$\mathbf{A} \in \mathbb{R}^{n \times c \times h \times w} \quad \text{(NCHW format)}$$

with a filter bank of m 2D filters of size $f_h \times f_w$ for the c channels

$$\mathbf{F} \in \mathbb{R}^{m \times c \times f_h \times f_w}.$$

Convolution based on BLAS (GEMM)

The matrix-matrix multiplication in the BLAS Level 3 (`GEMM` = GEneral matrix Matrix-Matrix multiplication) is fast.

Can we use `GEMM` for convolution? The task is to compute the sum over all channels c of a 2D image of size $h \times w$ for a minibatch n of images, here

$$\mathbf{A} \in \mathbb{R}^{n \times c \times h \times w} \quad \text{(NCHW format)}$$

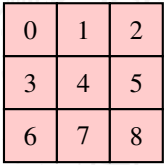
with a filter bank of m 2D filters of size $f_h \times f_w$ for the c channels

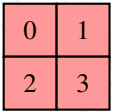
$$\mathbf{F} \in \mathbb{R}^{m \times c \times f_h \times f_w}.$$

We show that the above can be restated as matrix-matrix multiplication. We start with a single image of size 3×3 with three channels, namely $\mathbf{A} \in \mathbb{R}^{3 \times 3 \times 3}$ with a red, a green, and a blue channel, and a single filter over all channels, i.e., $\mathbf{F} \in \mathbb{R}^{3 \times 2 \times 2}$.

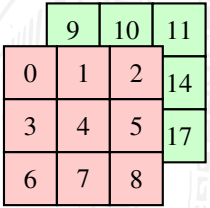
As second example we consider n images and m feature maps.

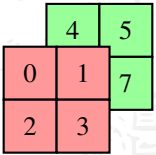
im2col: one image, one feature map



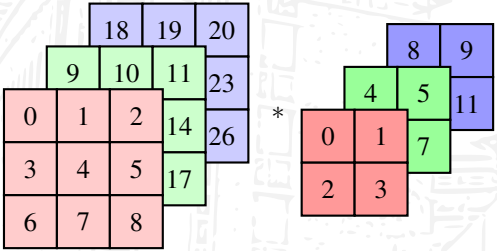


im2col: one image, one feature map

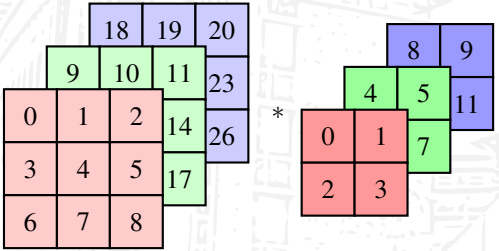




im2col: one image, one feature map

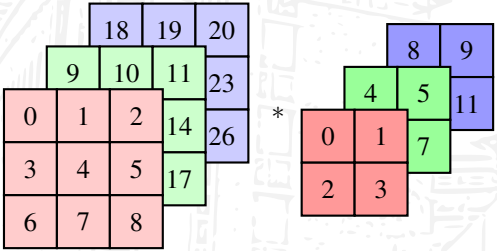


im2col: one image, one feature map



0	1	3	4
1	2	4	5
3	4	6	7
4	5	7	8

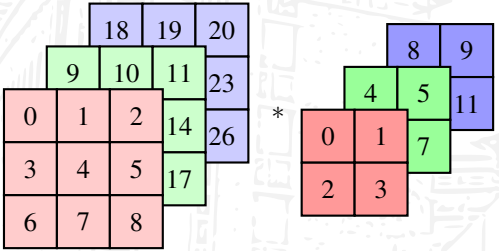
im2col: one image, one feature map



0	1	3	4
1	2	4	5
3	4	6	7
4	5	7	8

9	10	12	13
10	11	13	14
12	13	15	16
13	14	16	17

im2col: one image, one feature map

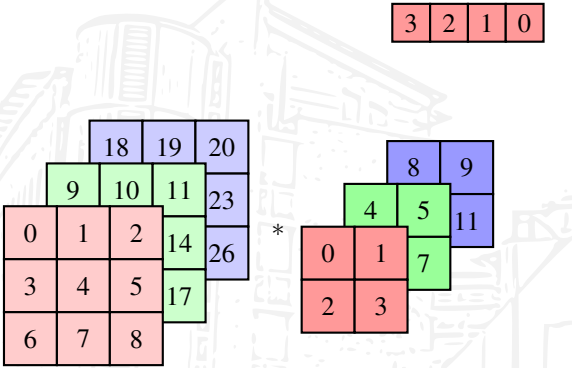


0	1	3	4
1	2	4	5
3	4	6	7
4	5	7	8

9	10	12	13
10	11	13	14
12	13	15	16
13	14	16	17

18	19	21	22
19	20	22	23
21	22	24	25
22	23	25	26

im2col: one image, one feature map

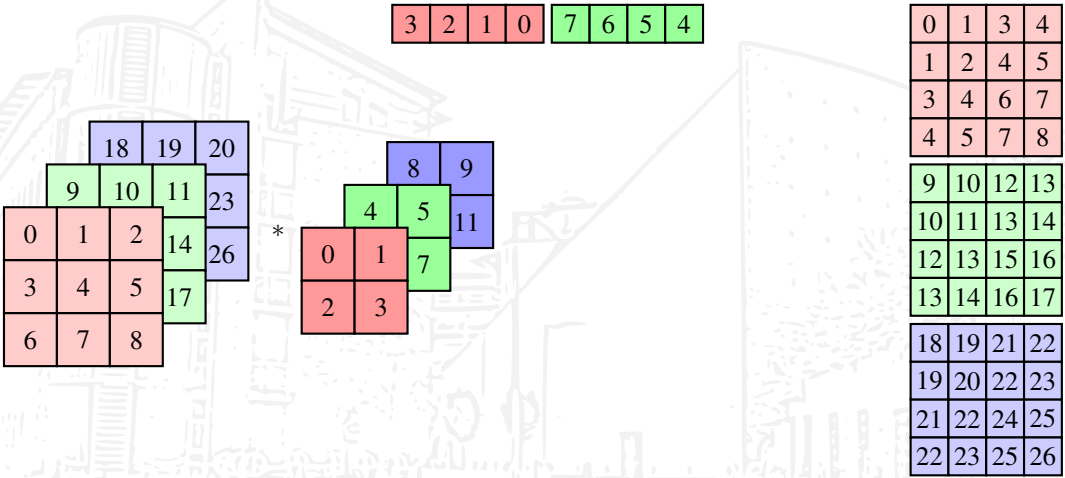


0	1	3	4
1	2	4	5
3	4	6	7
4	5	7	8

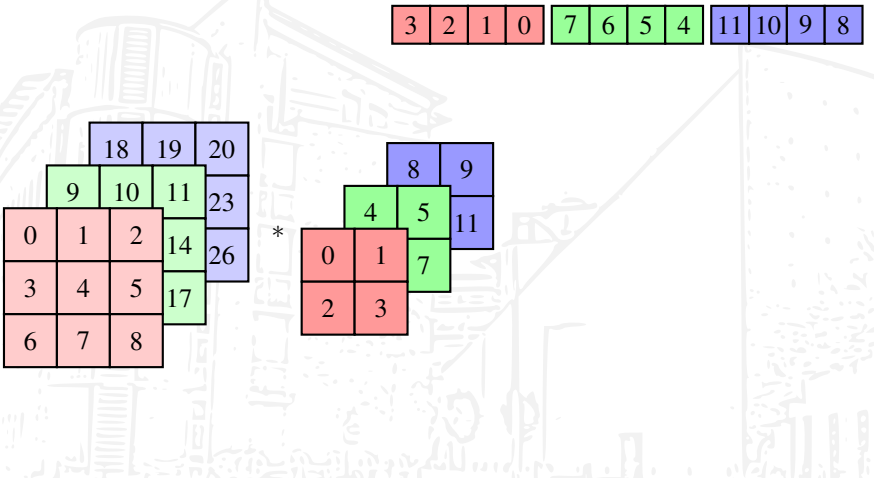
9	10	12	13
10	11	13	14
12	13	15	16
13	14	16	17

18	19	21	22
19	20	22	23
21	22	24	25
22	23	25	26

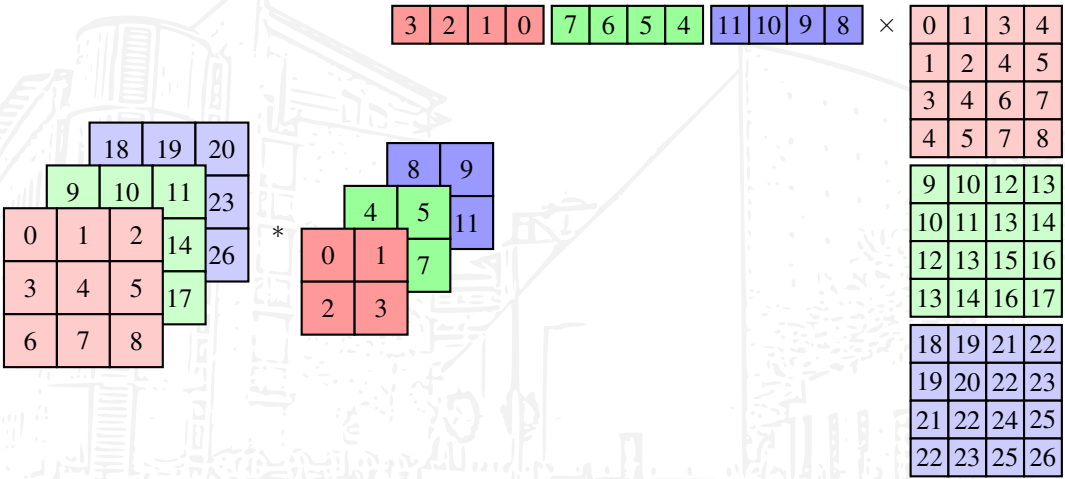
im2col: one image, one feature map



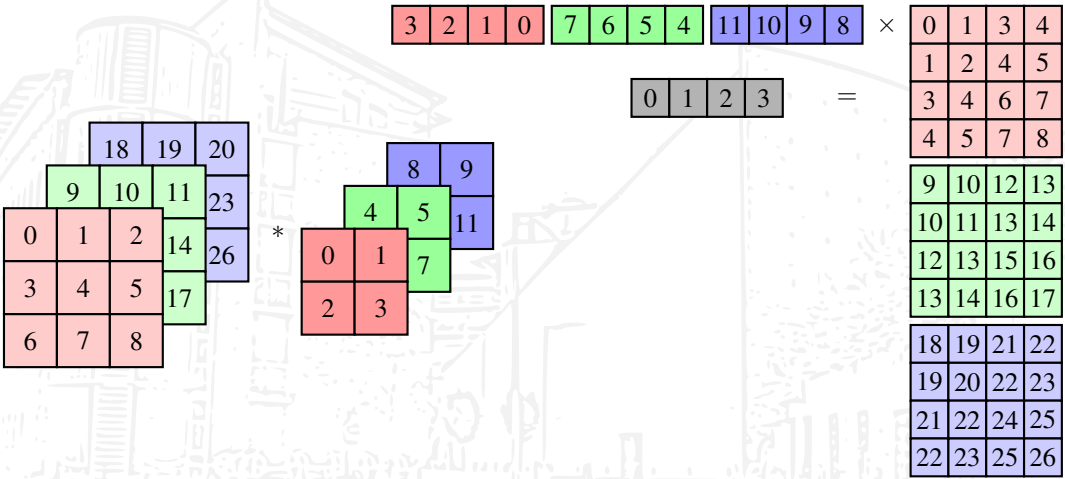
im2col: one image, one feature map



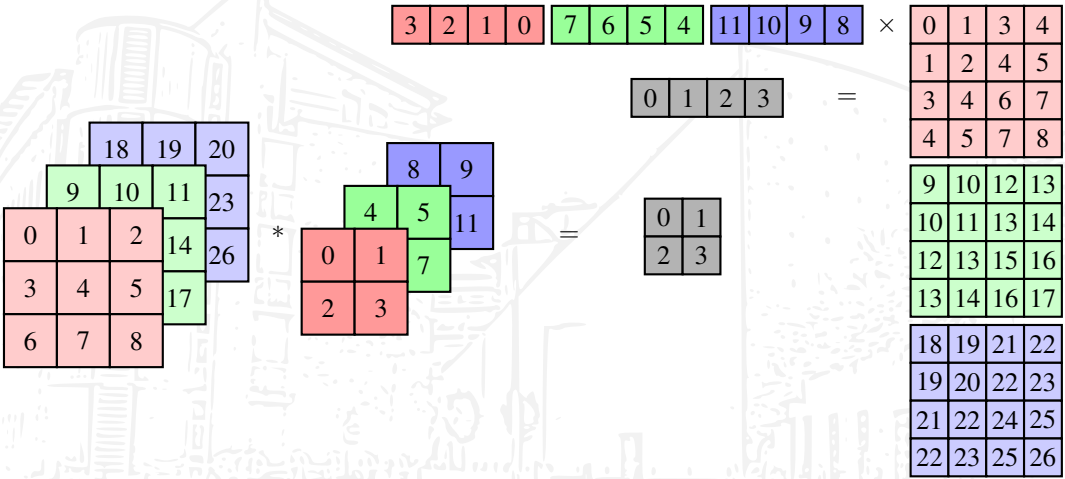
im2col: one image, one feature map



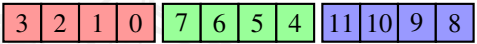
im2col: one image, one feature map

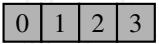


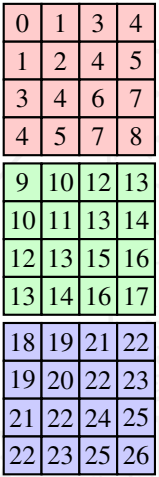
im2col: one image, one feature map



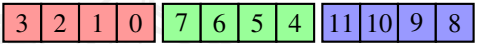
im2col: $n = 1$ images, $m = 1$ feature maps

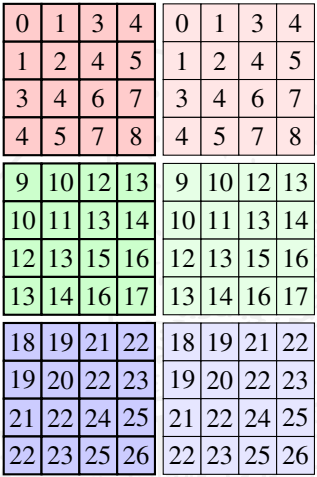




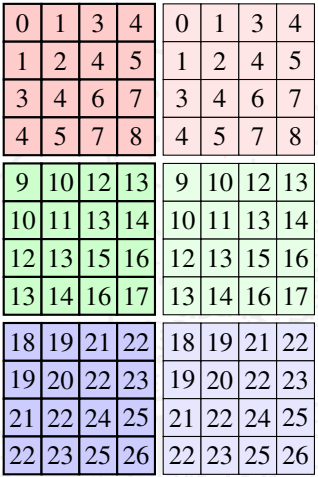


im2col: $n = 2$ images, $m = 1$ feature maps

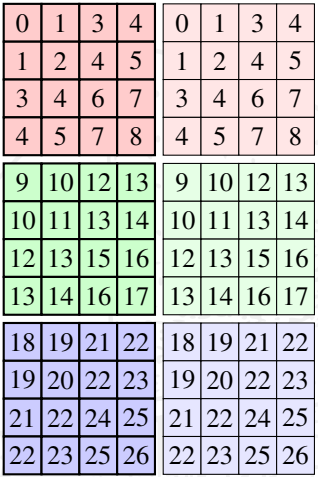


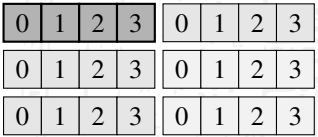


im2col: $n = 2$ images, $m = 2$ feature maps

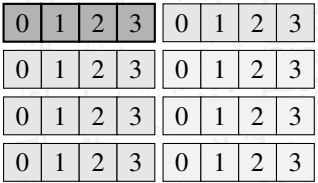


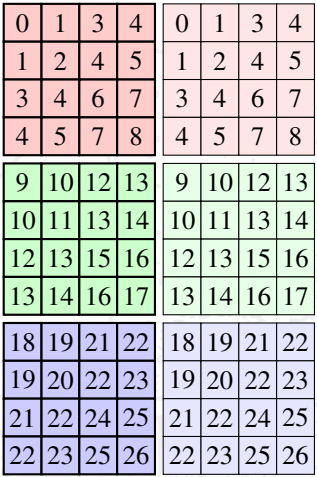
im2col: $n = 2$ images, $m = 3$ feature maps



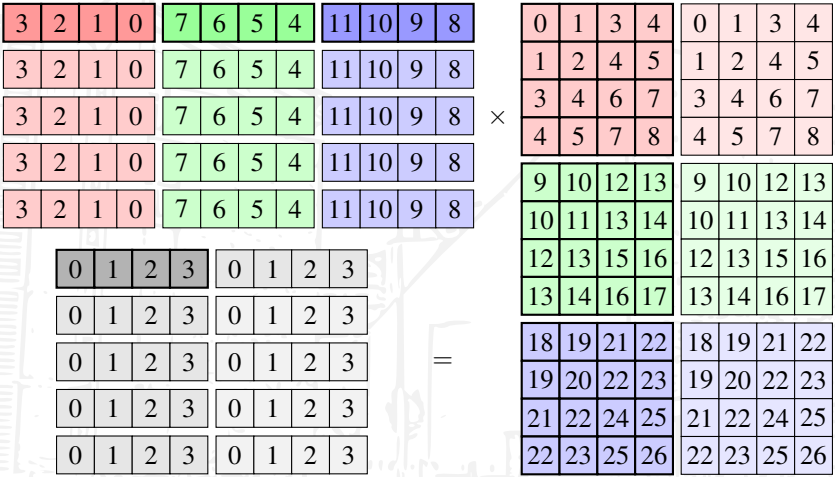


im2col: $n = 2$ images, $m = 4$ feature maps





im2col: $n = 2$ images, $m = 5$ feature maps



im2col: definition, generalizations, implementation

The matrix on the right-hand side is the so-called im2col matrix, obtained by im2col$(\mathbf{A}, f_h, f_w)$. The "inverse" mapping is known as col2im. The transpose results in the variants im2row and row2im.

im2col: definition, generalizations, implementation

The matrix on the right-hand side is the so-called im2col matrix, obtained by im2col$(\mathbf{A}, f_h, f_w)$. The "inverse" mapping is known as col2im. The transpose results in the variants im2row and row2im.

How to obtain im2col and col2im?

im2col: definition, generalizations, implementation

The matrix on the right-hand side is the so-called im2col matrix, obtained by im2col$(\mathbf{A}, f_h, f_w)$. The "inverse" mapping is known as col2im. The transpose results in the variants im2row and row2im.

How to obtain im2col and col2im?

We use advanced indexing: parts of

$$\texttt{A = np.arange(12).reshape(3, 4)} = \begin{pmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \end{pmatrix}$$

are obtained using

- `A[0, 0]`, `A[0, 1]` $\rightsquigarrow$ elements
- `A[0, :]`, `A[:, 0]` $\rightsquigarrow$ rows and columns
- `A[1:3, 1:3]` $\rightsquigarrow$ submatrices

im2col: definition, generalizations, implementation

The matrix on the right-hand side is the so-called im2col matrix, obtained by im2col$(\mathbf{A}, f_h, f_w)$. The "inverse" mapping is known as col2im. The transpose results in the variants im2row and row2im.

How to obtain im2col and col2im?

We use advanced indexing: parts of

$$\texttt{A = np.arange(12).reshape(3, 4)} = \begin{pmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \end{pmatrix}$$

are obtained using

- `A[0, 0]`, `A[0, 1]` $\rightsquigarrow$ elements
- `A[0, :]`, `A[:, 0]` $\rightsquigarrow$ rows and columns
- `A[1:3, 1:3]` $\rightsquigarrow$ submatrices
- `A[np.arange(3), np.arange(3)]` $\rightsquigarrow$ diagonal as a vector

im2col: definition, generalizations, implementation

The matrix on the right-hand side is the so-called im2col matrix, obtained by im2col$(\mathbf{A}, f_h, f_w)$. The "inverse" mapping is known as col2im. The transpose results in the variants im2row and row2im.

How to obtain im2col and col2im?

We use advanced indexing: parts of

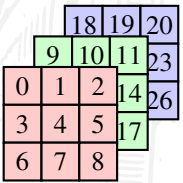
$$\texttt{A = np.arange(12).reshape(3, 4)} = \begin{pmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \end{pmatrix}$$

are obtained using

- `A[0, 0]`, `A[0, 1]` $\rightsquigarrow$ elements
- `A[0, :]`, `A[:, 0]` $\rightsquigarrow$ rows and columns
- `A[1:3, 1:3]` $\rightsquigarrow$ submatrices
- `A[np.arange(3), np.arange(3)]` $\rightsquigarrow$ diagonal as a vector
- `A[np.eye(2, dtype=int), np.eye(2, dtype=int)]` $\rightsquigarrow \begin{pmatrix} 5 & 0 \\ 0 & 5 \end{pmatrix}$

im2col: index matrices

This technique extends to nD arrays like $\mathbf{A} \in \mathbb{R}^{c \times h \times w}$, here for $c = 3$, $h = 3$, $w = 3$, $f_h = 2$, $f_w = 2$:



$$\Rightarrow \mathbf{A}\Big[$$

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1

2	2	2	2
2	2	2	2
2	2	2	2
2	2	2	2

,

0	0	1	1
0	0	1	1
1	1	2	2
1	1	2	2

0	0	1	1
0	0	1	1
1	1	2	2
1	1	2	2

0	0	1	1
0	0	1	1
1	1	2	2
1	1	2	2

,

0	1	0	1
1	2	1	2
0	1	0	1
1	2	1	2

0	1	0	1
1	2	1	2
0	1	0	1
1	2	1	2

0	1	0	1
1	2	1	2
0	1	0	1
1	2	1	2

$$\Big] =$$

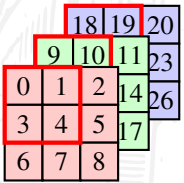
0	1	3	4
1	2	4	5
3	4	6	7
4	5	7	8

9	10	12	13
10	11	13	14
12	13	15	16
13	14	16	17

18	19	21	22
19	20	22	23
21	22	24	25
22	23	25	26

im2col: index matrices

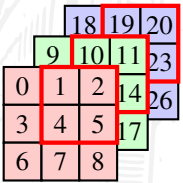
This technique extends to nD arrays like $\mathbf{A} \in \mathbb{R}^{c \times h \times w}$, here for $c = 3$, $h = 3$, $w = 3$, $f_h = 2$, $f_w = 2$:



$$\Rightarrow \mathbf{A}\Big[\begin{matrix} 0&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&0 \end{matrix} \begin{matrix} 1&1&1&1\\1&1&1&1\\1&1&1&1\\1&1&1&1 \end{matrix} \begin{matrix} 2&2&2&2\\2&2&2&2\\2&2&2&2\\2&2&2&2 \end{matrix} , \begin{matrix} 0&0&1&1\\0&0&1&1\\1&1&2&2\\1&1&2&2 \end{matrix} \begin{matrix} 0&0&1&1\\0&0&1&1\\1&1&2&2\\1&1&2&2 \end{matrix} \begin{matrix} 0&0&1&1\\0&0&1&1\\1&1&2&2\\1&1&2&2 \end{matrix} , \begin{matrix} 0&1&0&1\\1&2&1&2\\0&1&0&1\\1&2&1&2 \end{matrix} \begin{matrix} 0&1&0&1\\1&2&1&2\\0&1&0&1\\1&2&1&2 \end{matrix} \begin{matrix} 0&1&0&1\\1&2&1&2\\0&1&0&1\\1&2&1&2 \end{matrix} \Big] = \begin{matrix} 0&1&3&4\\1&2&4&5\\3&4&6&7\\4&5&7&8 \end{matrix} \begin{matrix} 9&10&12&13\\10&11&13&14\\12&13&15&16\\13&14&16&17 \end{matrix} \begin{matrix} 18&19&21&22\\19&20&22&23\\21&22&24&25\\22&23&25&26 \end{matrix}$$

im2col: index matrices

This technique extends to nD arrays like $\mathbf{A} \in \mathbb{R}^{c \times h \times w}$, here for $c = 3$, $h = 3$, $w = 3$, $f_h = 2$, $f_w = 2$:



$$\Rightarrow \mathbf{A}\Big[$$

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0
1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1
2	2	2	2
2	2	2	2
2	2	2	2
2	2	2	2

,

0	0	1	1
0	0	1	1
1	1	2	2
1	1	2	2
0	0	1	1
0	0	1	1
1	1	2	2
1	1	2	2
0	0	1	1
0	0	1	1
1	1	2	2
1	1	2	2

,

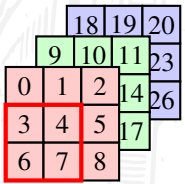
0	1	0	1
1	2	1	2
0	1	0	1
1	2	1	2
0	1	0	1
1	2	1	2
0	1	0	1
1	2	1	2
0	1	0	1
1	2	1	2
0	1	0	1
1	2	1	2

$$\Big] =$$

0	1	3	4
1	2	4	5
3	4	6	7
4	5	7	8
9	10	12	13
10	11	13	14
12	13	15	16
13	14	16	17
18	19	21	22
19	20	22	23
21	22	24	25
22	23	25	26

im2col: index matrices

This technique extends to nD arrays like $\mathbf{A} \in \mathbb{R}^{c \times h \times w}$, here for $c = 3$, $h = 3$, $w = 3$, $f_h = 2$, $f_w = 2$:



$\Rightarrow \mathbf{A}\Big[$

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0
1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1
2	2	2	2
2	2	2	2
2	2	2	2
2	2	2	2

,

0	0	1	1
0	0	1	1
1	1	2	2
1	1	2	2
0	0	1	1
0	0	1	1
1	1	2	2
1	1	2	2
0	0	1	1
0	0	1	1
1	1	2	2
1	1	2	2

,

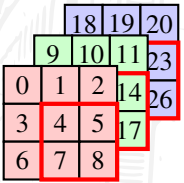
0	1	0	1
1	2	1	2
0	1	0	1
1	2	1	2
0	1	0	1
1	2	1	2
0	1	0	1
1	2	1	2
0	1	0	1
1	2	1	2
0	1	0	1
1	2	1	2

$\Big] =$

0	1	3	4
1	2	4	5
3	4	6	7
4	5	7	8
9	10	12	13
10	11	13	14
12	13	15	16
13	14	16	17
18	19	21	22
19	20	22	23
21	22	24	25
22	23	25	26

im2col: index matrices

This technique extends to nD arrays like $\mathbf{A} \in \mathbb{R}^{c \times h \times w}$, here for $c = 3$, $h = 3$, $w = 3$, $f_h = 2$, $f_w = 2$:



$$\Rightarrow \mathbf{A}\Big[$$

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1

2	2	2	2
2	2	2	2
2	2	2	2
2	2	2	2

,

0	0	1	1
0	0	1	1
1	1	2	2
1	1	2	2

0	0	1	1
0	0	1	1
1	1	2	2
1	1	2	2

0	0	1	1
0	0	1	1
1	1	2	2
1	1	2	2

,

0	1	0	1
1	2	1	2
0	1	0	1
1	2	1	2

0	1	0	1
1	2	1	2
0	1	0	1
1	2	1	2

0	1	0	1
1	2	1	2
0	1	0	1
1	2	1	2

$$\Big] =$$

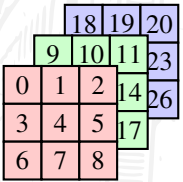
0	1	3	4
1	2	4	5
3	4	6	7
4	5	7	8

9	10	12	13
10	11	13	14
12	13	15	16
13	14	16	17

18	19	21	22
19	20	22	23
21	22	24	25
22	23	25	26

im2col: index matrices/vectors

This technique extends to nD arrays like $\mathbf{A} \in \mathbb{R}^{c \times h \times w}$, here for $c = 3$, $h = 3$, $w = 3$, $f_h = 2$, $f_w = 2$:



$$\Rightarrow \mathbf{A}\Big[\begin{array}{|c|c|c|c|}\hline 0&0&0&0\\\hline 1&1&1&1\\\hline 2&2&2&2\\\hline\end{array}, \begin{array}{|c|c|c|c|}\hline 0&0&1&1\\\hline 0&0&1&1\\\hline 1&1&2&2\\\hline 1&1&2&2\\\hline\end{array}, \begin{array}{|c|c|c|c|}\hline 0&1&0&1\\\hline 1&2&1&2\\\hline 0&1&0&1\\\hline 1&2&1&2\\\hline\end{array}\Big] = \begin{array}{|c|c|c|c|}\hline 0&1&3&4\\\hline 1&2&4&5\\\hline 3&4&6&7\\\hline 4&5&7&8\\\hline\end{array}\begin{array}{|c|c|c|c|}\hline 9&10&12&13\\\hline 10&11&13&14\\\hline 12&13&15&16\\\hline 13&14&16&17\\\hline\end{array}\begin{array}{|c|c|c|c|}\hline 18&19&21&22\\\hline 19&20&22&23\\\hline 21&22&24&25\\\hline 22&23&25&26\\\hline\end{array}$$

im2col: index matrices/vectors

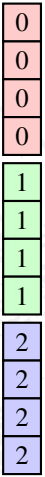
Image: (c, h, w), filter: (f_h, f_w), output: $(z_h = h - f_h + 1, z_w = w - f_w + 1)$

im2col: index matrices/vectors

Image: (c, h, w), filter: (f_h, f_w), output: $(z_h = h - f_h + 1, z_w = w - f_w + 1)$

Computation of indices `ch`, `i`, `j` using `np.repeat` and `np.tile`:

channel index: `np.repeat(np.arange(c), fh * fw).reshape(-1, 1)`



im2col: index matrices/vectors

Image: (c, h, w), filter: (f_h, f_w), output: $(z_h = h - f_h + 1, z_w = w - f_w + 1)$

Computation of indices `ch`, `i`, `j` using `np.repeat` and `np.tile`:

channel index:
```
np.repeat(np.arange(c), fh * fw).reshape(-1, 1)
```

row index:
```
i0 = np.tile(np.repeat(np.arange(fh), fw), c)
i1 = np.repeat(np.arange(zh), zw)
i = i0.reshape(-1, 1) + i1.reshape(1, -1)
```

0	0	1	1
0	0	1	1
1	1	2	2
1	1	2	2

0	0	1	1
0	0	1	1
1	1	2	2
1	1	2	2

0	0	1	1
0	0	1	1
1	1	2	2
1	1	2	2

im2col: index matrices/vectors

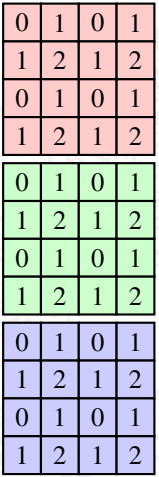
Image: (c, h, w), filter: (f_h, f_w), output: $(z_h = h - f_h + 1, z_w = w - f_w + 1)$

Computation of indices `ch`, `i`, `j` using `np.repeat` and `np.tile`:

channel index:
```
np.repeat(np.arange(c), fh * fw).reshape(-1, 1)
```

row index:
```
i0 = np.tile(np.repeat(np.arange(fh), fw), c)
i1 = np.repeat(np.arange(zh), zw)
i = i0.reshape(-1, 1) + i1.reshape(1, -1)
```

column index:
```
j0 = np.tile(np.arange(fw), fh * c)
j1 = np.tile(np.arange(zw), zh)
j = j0.reshape(-1, 1) + j1.reshape(1, -1)
```



im2col: index matrices/vectors

Image: (c, h, w), filter: (f_h, f_w), output: $(z_h = h - f_h + 1, z_w = w - f_w + 1)$

Computation of indices `ch`, `i`, `j` using `np.repeat` and `np.tile`:

channel index:
```
np.repeat(np.arange(c), fh * fw).reshape(-1, 1)
```

row index:
```
i0 = np.tile(np.repeat(np.arange(fh), fw), c)
i1 = np.repeat(np.arange(zh), zw)
i = i0.reshape(-1, 1) + i1.reshape(1, -1)
```

column index:
```
j0 = np.tile(np.arange(fw), fh * c)
j1 = np.tile(np.arange(zw), zh)
j = j0.reshape(-1, 1) + j1.reshape(1, -1)
```

Compute $\mathbf{A}_{\text{col}} =$ `np.concatenate(`$\mathbf{A}$`[:, ch, i, j], axis=1)`

0	1	3	4
1	2	4	5
3	4	6	7
4	5	7	8

9	10	12	13
10	11	13	14
12	13	15	16
13	14	16	17

18	19	21	22
19	20	22	23
21	22	24	25
22	23	25	26

im2col: backpropagation & update in matrix form

For $\mathbf{A} \in \mathbb{R}^{n \times c \times h \times w}$ and $\mathbf{F} \in \mathbb{R}^{m \times c \times f_h \times f_w}$, $\mathbf{Z} \in \mathbb{R}^{n \times m \times z_h \times z_w}$, where $z_h = h - f_h + 1$, $z_w = w - f_w + 1$. Thus,

$$\boldsymbol{\Delta} := \frac{\partial C}{\partial \mathbf{Z}} \in \mathbb{R}^{n \times m \times z_h \times z_w}.$$

im2col: backpropagation & update in matrix form

For $\mathbf{A} \in \mathbb{R}^{n \times c \times h \times w}$ and $\mathbf{F} \in \mathbb{R}^{m \times c \times f_h \times f_w}$, $\mathbf{Z} \in \mathbb{R}^{n \times m \times z_h \times z_w}$, where $z_h = h - f_h + 1$, $z_w = w - f_w + 1$. Thus,

$$\boldsymbol{\Delta} := \frac{\partial C}{\partial \mathbf{Z}} \in \mathbb{R}^{n \times m \times z_h \times z_w}.$$

In feedforward we work w/ matrix times matrix like in `FeedforwardNet`,

$$\mathbf{F} \rightsquigarrow \mathbf{F}_{\text{row}}, \quad \mathbf{A} \rightsquigarrow \mathbf{A}_{\text{col}}, \quad \mathbf{Z}_{\text{mat}} := \mathbf{F}_{\text{row}} \cdot \mathbf{A}_{\text{col}} + \mathbf{b}\mathbf{e}^\mathsf{T} \rightsquigarrow \mathbf{Z}.$$

im2col: backpropagation & update in matrix form

For $\mathbf{A} \in \mathbb{R}^{n \times c \times h \times w}$ and $\mathbf{F} \in \mathbb{R}^{m \times c \times f_h \times f_w}$, $\mathbf{Z} \in \mathbb{R}^{n \times m \times z_h \times z_w}$, where $z_h = h - f_h + 1$, $z_w = w - f_w + 1$. Thus,

$$\boldsymbol{\Delta} := \frac{\partial C}{\partial \mathbf{Z}} \in \mathbb{R}^{n \times m \times z_h \times z_w}.$$

In feedforward we work w/ matrix times matrix like in `FeedforwardNet`,

$$\mathbf{F} \rightsquigarrow \mathbf{F}_{\text{row}}, \quad \mathbf{A} \rightsquigarrow \mathbf{A}_{\text{col}}, \quad \mathbf{Z}_{\text{mat}} := \mathbf{F}_{\text{row}} \cdot \mathbf{A}_{\text{col}} + \mathbf{b}\mathbf{e}^{\mathsf{T}} \rightsquigarrow \mathbf{Z}.$$

Thus, backpropagation & update in the matrix formulation are given by

$$\boldsymbol{\Delta} \rightsquigarrow \boldsymbol{\Delta}_{\text{mat}}, \quad \texttt{df\_row} := \frac{\partial C}{\partial \mathbf{F}_{\text{row}}} = \boldsymbol{\Delta}_{\text{mat}} \cdot \mathbf{A}_{\text{col}}^{\mathsf{T}}, \quad \texttt{da\_col} := \frac{\partial C}{\partial \mathbf{A}_{\text{col}}} = \mathbf{F}_{\text{row}}^{\mathsf{T}} \cdot \boldsymbol{\Delta}_{\text{mat}}.$$

im2col: backpropagation & update in matrix form

For $\mathbf{A} \in \mathbb{R}^{n \times c \times h \times w}$ and $\mathbf{F} \in \mathbb{R}^{m \times c \times f_h \times f_w}$, $\mathbf{Z} \in \mathbb{R}^{n \times m \times z_h \times z_w}$, where $z_h = h - f_h + 1$, $z_w = w - f_w + 1$. Thus,

$$\boldsymbol{\Delta} := \frac{\partial C}{\partial \mathbf{Z}} \in \mathbb{R}^{n \times m \times z_h \times z_w}.$$

In feedforward we work w/ matrix times matrix like in `FeedforwardNet`,

$$\mathbf{F} \rightsquigarrow \mathbf{F}_{\text{row}}, \quad \mathbf{A} \rightsquigarrow \mathbf{A}_{\text{col}}, \quad \mathbf{Z}_{\text{mat}} := \mathbf{F}_{\text{row}} \cdot \mathbf{A}_{\text{col}} + \mathbf{b}\mathbf{e}^{\mathsf{T}} \rightsquigarrow \mathbf{Z}.$$

Thus, backpropagation & update in the matrix formulation are given by

$$\boldsymbol{\Delta} \rightsquigarrow \boldsymbol{\Delta}_{\text{mat}}, \quad \texttt{df\_row} := \frac{\partial C}{\partial \mathbf{F}_{\text{row}}} = \boldsymbol{\Delta}_{\text{mat}} \cdot \mathbf{A}_{\text{col}}^{\mathsf{T}}, \quad \texttt{da\_col} := \frac{\partial C}{\partial \mathbf{A}_{\text{col}}} = \mathbf{F}_{\text{row}}^{\mathsf{T}} \cdot \boldsymbol{\Delta}_{\text{mat}}.$$

We need the transformations $\dfrac{\partial C}{\partial \mathbf{F}_{\text{row}}} \rightsquigarrow \dfrac{\partial C}{\partial \mathbf{F}} =: \texttt{df}$ and $\dfrac{\partial C}{\partial \mathbf{A}_{\text{col}}} \rightsquigarrow \dfrac{\partial C}{\partial \mathbf{A}} =: \texttt{da}$.

im2col: backpropagation & update in matrix form

For $\mathbf{A} \in \mathbb{R}^{n \times c \times h \times w}$ and $\mathbf{F} \in \mathbb{R}^{m \times c \times f_h \times f_w}$, $\mathbf{Z} \in \mathbb{R}^{n \times m \times z_h \times z_w}$, where $z_h = h - f_h + 1$, $z_w = w - f_w + 1$. Thus,

$$\boldsymbol{\Delta} := \frac{\partial C}{\partial \mathbf{Z}} \in \mathbb{R}^{n \times m \times z_h \times z_w}.$$

In feedforward we work w/ matrix times matrix like in `FeedforwardNet`,

$$\mathbf{F} \rightsquigarrow \mathbf{F}_{\text{row}}, \quad \mathbf{A} \rightsquigarrow \mathbf{A}_{\text{col}}, \quad \mathbf{Z}_{\text{mat}} := \mathbf{F}_{\text{row}} \cdot \mathbf{A}_{\text{col}} + \mathbf{b}\mathbf{e}^\mathsf{T} \rightsquigarrow \mathbf{Z}.$$

Thus, backpropagation & update in the matrix formulation are given by

$$\boldsymbol{\Delta} \rightsquigarrow \boldsymbol{\Delta}_{\text{mat}}, \quad \texttt{df\_row} := \frac{\partial C}{\partial \mathbf{F}_{\text{row}}} = \boldsymbol{\Delta}_{\text{mat}} \cdot \mathbf{A}_{\text{col}}^\mathsf{T}, \quad \texttt{da\_col} := \frac{\partial C}{\partial \mathbf{A}_{\text{col}}} = \mathbf{F}_{\text{row}}^\mathsf{T} \cdot \boldsymbol{\Delta}_{\text{mat}}.$$

We need the transformations $\dfrac{\partial C}{\partial \mathbf{F}_{\text{row}}} \rightsquigarrow \dfrac{\partial C}{\partial \mathbf{F}} =: \texttt{df}$ and $\dfrac{\partial C}{\partial \mathbf{A}_{\text{col}}} \rightsquigarrow \dfrac{\partial C}{\partial \mathbf{A}} =: \texttt{da}$.

Simple transformations (blue): exercise, col2im (red): next slides

im2col: transformed backpropagation & update

The mappings

$$\mathbf{F} \rightsquigarrow \mathbf{F}_{\text{row}}, \quad \mathbf{Z}_{\text{mat}} \rightsquigarrow \mathbf{Z}, \quad \boldsymbol{\Delta} \rightsquigarrow \boldsymbol{\Delta}_{\text{mat}}, \quad \texttt{df\_row} \rightsquigarrow \texttt{df}$$

map the same number of elements to a different shape. This can be accomplished using `np.reshape` and `np.transpose` (w/ flipping of filters).

im2col: transformed backpropagation & update

The mappings

$$\mathbf{F} \rightsquigarrow \mathbf{F}_{\text{row}}, \quad \mathbf{Z}_{\text{mat}} \rightsquigarrow \mathbf{Z}, \quad \mathbf{\Delta} \rightsquigarrow \mathbf{\Delta}_{\text{mat}}, \quad \texttt{df\_row} \rightsquigarrow \texttt{df}$$

map the same number of elements to a different shape. This can be accomplished using `np.reshape` and `np.transpose` (w/ flipping of filters).

The mapping `da_col` $\rightsquigarrow$ `da` changes the number of elements. We have to sum some elements due to the weight sharing. If all entries of `da_col` are one, then for $h = w = 3, f_h = f_w = 2$

$$\texttt{da}[i,j,:,:] = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}.$$

im2col: transformed backpropagation & update

The mappings

$$\mathbf{F} \rightsquigarrow \mathbf{F}_{\text{row}}, \quad \mathbf{Z}_{\text{mat}} \rightsquigarrow \mathbf{Z}, \quad \boldsymbol{\Delta} \rightsquigarrow \boldsymbol{\Delta}_{\text{mat}}, \quad \texttt{df\_row} \rightsquigarrow \texttt{df}$$

map the same number of elements to a different shape. This can be accomplished using `np.reshape` and `np.transpose` (w/ flipping of filters).

The mapping `da_col` $\rightsquigarrow$ `da` changes the number of elements. We have to sum some elements due to the weight sharing. If all entries of `da_col` are one, then for $h = w = 3, f_h = f_w = 2$

$$\texttt{da}[i,j,:,:] = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}.$$

This follows by connectivity:

$$\mathbf{A} = \begin{array}{|c|c|c|} \hline 0 & 1 & 2 \\ \hline 3 & 4 & 5 \\ \hline 6 & 7 & 8 \\ \hline \end{array},$$

im2col: transformed backpropagation & update

The mappings

$$\mathbf{F} \rightsquigarrow \mathbf{F}_{\text{row}}, \quad \mathbf{Z}_{\text{mat}} \rightsquigarrow \mathbf{Z}, \quad \boldsymbol{\Delta} \rightsquigarrow \boldsymbol{\Delta}_{\text{mat}}, \quad \texttt{df\_row} \rightsquigarrow \texttt{df}$$

map the same number of elements to a different shape. This can be accomplished using `np.reshape` and `np.transpose` (w/ flipping of filters).

The mapping `da_col` $\rightsquigarrow$ `da` changes the number of elements. We have to sum some elements due to the weight sharing. If all entries of `da_col` are one, then for $h = w = 3, f_h = f_w = 2$

$$\texttt{da}[i,j,:,:] = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}.$$

This follows by connectivity:

$$\mathbf{A} = \begin{array}{|c|c|c|}\hline 0 & 1 & 2 \\ \hline 3 & 4 & 5 \\ \hline 6 & 7 & 8 \\ \hline \end{array}, \qquad \frac{\partial C}{\partial \mathbf{A}} = \begin{array}{|c|c|c|}\hline 1 & 1 & 0 \\ \hline 1 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array}.$$

im2col: transformed backpropagation & update

The mappings

$$\mathbf{F} \rightsquigarrow \mathbf{F}_{\text{row}}, \quad \mathbf{Z}_{\text{mat}} \rightsquigarrow \mathbf{Z}, \quad \boldsymbol{\Delta} \rightsquigarrow \boldsymbol{\Delta}_{\text{mat}}, \quad \texttt{df\_row} \rightsquigarrow \texttt{df}$$

map the same number of elements to a different shape. This can be accomplished using `np.reshape` and `np.transpose` (w/ flipping of filters).

The mapping `da_col` $\rightsquigarrow$ `da` changes the number of elements. We have to sum some elements due to the weight sharing. If all entries of `da_col` are one, then for $h = w = 3, f_h = f_w = 2$

$$\texttt{da}[i,j,:,:] = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}.$$

This follows by connectivity:

$$\mathbf{A} = \begin{array}{|c|c|c|} \hline 0 & 1 & 2 \\ \hline 3 & 4 & 5 \\ \hline 6 & 7 & 8 \\ \hline \end{array}, \qquad \frac{\partial C}{\partial \mathbf{A}} = \begin{array}{|c|c|c|} \hline 1 & 2 & 1 \\ \hline 1 & 2 & 1 \\ \hline 0 & 0 & 0 \\ \hline \end{array}.$$

im2col: transformed backpropagation & update

The mappings

$$\mathbf{F} \rightsquigarrow \mathbf{F}_{\text{row}}, \quad \mathbf{Z}_{\text{mat}} \rightsquigarrow \mathbf{Z}, \quad \boldsymbol{\Delta} \rightsquigarrow \boldsymbol{\Delta}_{\text{mat}}, \quad \texttt{df\_row} \rightsquigarrow \texttt{df}$$

map the same number of elements to a different shape. This can be accomplished using `np.reshape` and `np.transpose` (w/ flipping of filters).

The mapping `da_col` $\rightsquigarrow$ `da` changes the number of elements. We have to sum some elements due to the weight sharing. If all entries of `da_col` are one, then for $h = w = 3, f_h = f_w = 2$

$$\texttt{da}[i,j,:,:] = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}.$$

This follows by connectivity:

$$\mathbf{A} = \begin{array}{|c|c|c|} \hline 0 & 1 & 2 \\ \hline 3 & 4 & 5 \\ \hline 6 & 7 & 8 \\ \hline \end{array}, \qquad \frac{\partial C}{\partial \mathbf{A}} = \begin{array}{|c|c|c|} \hline 1 & 2 & 1 \\ \hline 2 & 3 & 1 \\ \hline 1 & 1 & 0 \\ \hline \end{array}.$$

im2col: transformed backpropagation & update

The mappings

$$\mathbf{F} \rightsquigarrow \mathbf{F}_{\text{row}}, \quad \mathbf{Z}_{\text{mat}} \rightsquigarrow \mathbf{Z}, \quad \boldsymbol{\Delta} \rightsquigarrow \boldsymbol{\Delta}_{\text{mat}}, \quad \texttt{df\_row} \rightsquigarrow \texttt{df}$$

map the same number of elements to a different shape. This can be accomplished using `np.reshape` and `np.transpose` (w/ flipping of filters).

The mapping `da_col` $\rightsquigarrow$ `da` changes the number of elements. We have to sum some elements due to the weight sharing. If all entries of `da_col` are one, then for $h = w = 3, f_h = f_w = 2$

$$\texttt{da}[i,j,:,:] = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}.$$

This follows by connectivity:

$$\mathbf{A} = \begin{array}{|c|c|c|} \hline 0 & 1 & 2 \\ \hline 3 & 4 & 5 \\ \hline 6 & 7 & 8 \\ \hline \end{array}, \qquad \frac{\partial C}{\partial \mathbf{A}} = \begin{array}{|c|c|c|} \hline 1 & 2 & 1 \\ \hline 2 & 4 & 2 \\ \hline 1 & 2 & 1 \\ \hline \end{array}.$$

im2col: the problem w/ reversal of im2col

We would like to use our indexing w/ the matrices `i` and `j`,

$$\mathtt{i} = \begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 2 & 2 \\ 1 & 1 & 2 & 2 \end{pmatrix}, \quad \mathtt{j} = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 2 & 1 & 2 \\ 0 & 1 & 0 & 1 \\ 1 & 2 & 1 & 2 \end{pmatrix},$$

im2col: the problem w/ reversal of im2col

We can not directly use our indexing w/ the matrices `i` and `j`,

$$\texttt{i} = \begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 2 & 2 \\ 1 & 1 & 2 & 2 \end{pmatrix}, \quad \texttt{j} = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 2 & 1 & 2 \\ 0 & 1 & 0 & 1 \\ 1 & 2 & 1 & 2 \end{pmatrix},$$

as due to buffering

```
da_test = np.zeros((3, 3))
da_test[i, j] += 1
```

results in

$$\texttt{da\_test} = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} \neq \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix} = \texttt{da}.$$

im2col: col2im

This problem is solved by `np.add.at`:

```
da_test = np.zeros((3, 3))
np.add.at(da_test, (i, j), 1)
```

results in

$$\texttt{da\_test} = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix} = \texttt{da}.$$

im2col: col2im

This problem is solved by `np.add.at`:

```
da_test = np.zeros((3, 3))
np.add.at(da_test, (i, j), 1)
```

results in

$$\texttt{da\_test} = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix} = \texttt{da}.$$

col2im is based on:

```
da = np.zeros((n, c, h, w))
da_col_split = np.array(np.hsplit(da_col, n))
np.add.at(da, (slice(None), ch, i, j), da_col_split)
```

im2col: convolutions as dense layers

im2col: convolutions like ordinary dense layers (w/ reshaping and im2col/col2im transformations):

im2col: convolutions as dense layers

im2col: convolutions like ordinary dense layers (w/ reshaping and im2col/col2im transformations):

Feedforward (w/o activation):

reshape: $\mathbf{F} \rightsquigarrow \mathbf{F}_{\text{row}}$ $\rightsquigarrow$ exercise

im2col: $\mathbf{A} \rightsquigarrow \mathbf{A}_{\text{col}}$

GEMM: $\mathbf{Z}_{\text{mat}} = \mathbf{F}_{\text{row}} \cdot \mathbf{A}_{\text{col}} + \mathbf{b}\mathbf{e}^{\top}$

reshape: $\mathbf{Z}_{\text{mat}} \rightsquigarrow \mathbf{Z}$ $\rightsquigarrow$ exercise

im2col: convolutions as dense layers

im2col: convolutions like ordinary dense layers (w/ reshaping and im2col/col2im transformations):

Feedforward (w/o activation):

- reshape: $\mathbf{F} \rightsquigarrow \mathbf{F}_{\text{row}}$ $\rightsquigarrow$ exercise
- im2col: $\mathbf{A} \rightsquigarrow \mathbf{A}_{\text{col}}$
- GEMM: $\mathbf{Z}_{\text{mat}} = \mathbf{F}_{\text{row}} \cdot \mathbf{A}_{\text{col}} + \mathbf{b}\mathbf{e}^{\mathsf{T}}$
- reshape: $\mathbf{Z}_{\text{mat}} \rightsquigarrow \mathbf{Z}$ $\rightsquigarrow$ exercise

Backpropagation (w/o activation):

- reshape: $\boldsymbol{\Delta} \rightsquigarrow \boldsymbol{\Delta}_{\text{mat}}$ $\rightsquigarrow$ exercise
- GEMM: $\texttt{da\_col} = \mathbf{F}_{\text{row}}^{\mathsf{T}} \cdot \boldsymbol{\Delta}_{\text{mat}}$
- col2im: $\texttt{da\_col} \rightsquigarrow \texttt{da}$

im2col: convolutions as dense layers

im2col: convolutions like ordinary dense layers (w/ reshaping and im2col/col2im transformations):

Feedforward (w/o activation):

- reshape: $\mathbf{F} \rightsquigarrow \mathbf{F}_{\text{row}}$ $\rightsquigarrow$ exercise
- im2col: $\mathbf{A} \rightsquigarrow \mathbf{A}_{\text{col}}$
- GEMM: $\mathbf{Z}_{\text{mat}} = \mathbf{F}_{\text{row}} \cdot \mathbf{A}_{\text{col}} + \mathbf{b}\mathbf{e}^\mathsf{T}$
- reshape: $\mathbf{Z}_{\text{mat}} \rightsquigarrow \mathbf{Z}$ $\rightsquigarrow$ exercise

Backpropagation (w/o activation):

- reshape: $\boldsymbol{\Delta} \rightsquigarrow \boldsymbol{\Delta}_{\text{mat}}$ $\rightsquigarrow$ exercise
- GEMM: $\texttt{da\_col} = \mathbf{F}_{\text{row}}^\mathsf{T} \cdot \boldsymbol{\Delta}_{\text{mat}}$
- col2im: $\texttt{da\_col} \rightsquigarrow \texttt{da}$

Update:

- sum:
```
db = np.sum(delta,
axis=(0, 2, 3))
```
- GEMM: $\texttt{df\_row} = \boldsymbol{\Delta}_{\text{mat}} \cdot \mathbf{A}_{\text{col}}^\mathsf{T}$
- reshape: $\texttt{df\_row} \rightsquigarrow \texttt{df}$ $\rightsquigarrow$ exercise

im2col: extensions, pros, & cons

Remark 1: im2col and col2im an be extended to deal with padding and strides.
Remark 2: max pooling can be computed via im2col with strides.

im2col: extensions, pros, & cons

Remark 1: im2col and col2im an be extended to deal with padding and strides.
Remark 2: max pooling can be computed via im2col with strides.

Pros: im2col has remarkable properties, namely:

- uses BLAS Level 3, thus it is fast
- the implementation is simplified a lot

im2col: extensions, pros, & cons

Remark 1: im2col and col2im an be extended to deal with padding and strides.
Remark 2: max pooling can be computed via im2col with strides.

Pros: im2col has remarkable properties, namely:

- uses BLAS Level 3, thus it is fast
- the implementation is simplified a lot

On the dark side we have as con:

- the im2col matrix uses a lot more memory, as all inner elements of $\mathbf{A}$ have to be stored k^2 times for filters of sizes $k \times k$

im2col: extensions, pros, & cons

Remark 1: im2col and col2im an be extended to deal with padding and strides.
Remark 2: max pooling can be computed via im2col with strides.

Pros: im2col has remarkable properties, namely:

- uses BLAS Level 3, thus it is fast
- the implementation is simplified a lot

On the dark side we have as con:

- the im2col matrix uses a lot more memory, as all inner elements of $\mathbf{A}$ have to be stored k^2 times for filters of sizes $k \times k$

im2col complements FFT nicely, as FFT works for large filters and im2col for small filters.

im2col: extensions, pros, & cons

Remark 1: im2col and col2im an be extended to deal with padding and strides.
Remark 2: max pooling can be computed via im2col with strides.

Pros: im2col has remarkable properties, namely:

- uses BLAS Level 3, thus it is fast
- the implementation is simplified a lot

On the dark side we have as con:

- the im2col matrix uses a lot more memory, as all inner elements of $\mathbf{A}$ have to be stored k^2 times for filters of sizes $k \times k$

im2col complements FFT nicely, as FFT works for large filters and im2col for small filters.

Can we do even better for small filters?

Winograd's minimal algorithm for $F(2,3)$

Winograd [11] derived minimal algorithms, e.g., a minimal algorithm for $F(2,3)$, i.e., partial cross-correlation of $\mathbf{x} \in \mathbb{R}^4$ with $\mathbf{f} \in \mathbb{R}^3$,

$$\begin{pmatrix} x_0 & x_1 & x_2 \\ x_1 & x_2 & x_3 \end{pmatrix} \begin{pmatrix} f_0 \\ f_1 \\ f_2 \end{pmatrix} = \begin{pmatrix} m_1 + m_2 + m_3 \\ m_2 - m_3 - m_4 \end{pmatrix},$$

Winograd's minimal algorithm for $F(2,3)$

Winograd [11] derived minimal algorithms, e.g., a minimal algorithm for $F(2,3)$, i.e., partial cross-correlation of $\mathbf{x} \in \mathbb{R}^4$ with $\mathbf{f} \in \mathbb{R}^3$,

$$\begin{pmatrix} x_0 & x_1 & x_2 \\ x_1 & x_2 & x_3 \end{pmatrix} \begin{pmatrix} f_0 \\ f_1 \\ f_2 \end{pmatrix} = \begin{pmatrix} m_1 + m_2 + m_3 \\ m_2 - m_3 - m_4 \end{pmatrix},$$

where

$$m_1 = (x_0 - x_2) \times f_0, \qquad m_2 = (x_1 + x_2) \times \frac{f_0 + f_1 + f_2}{2},$$

$$m_3 = (x_2 - x_1) \times \frac{f_0 - f_1 + f_2}{2}, \qquad m_4 = (x_1 - x_3) \times f_2$$

are obtained using $4 = 2 + 3 - 1$ multiplications (denoted by $\times$).

Winograd's minimal algorithm for $F(2,3)$

Winograd [11] derived minimal algorithms, e.g., a minimal algorithm for $F(2,3)$, i.e., partial cross-correlation of $\mathbf{x} \in \mathbb{R}^4$ with $\mathbf{f} \in \mathbb{R}^3$,

$$\begin{pmatrix} x_0 & x_1 & x_2 \\ x_1 & x_2 & x_3 \end{pmatrix} \begin{pmatrix} f_0 \\ f_1 \\ f_2 \end{pmatrix} = \begin{pmatrix} m_1 + m_2 + m_3 \\ m_2 - m_3 - m_4 \end{pmatrix},$$

where

$$m_1 = (x_0 - x_2) \times f_0, \qquad m_2 = (x_1 + x_2) \times \frac{f_0 + f_1 + f_2}{2},$$

$$m_3 = (x_2 - x_1) \times \frac{f_0 - f_1 + f_2}{2}, \qquad m_4 = (x_1 - x_3) \times f_2$$

are obtained using $4 = 2 + 3 - 1$ multiplications (denoted by $\times$).

Improvement of $1.5 = 6/4$ over direct cross-correlation, where six multiplications are used.

Matrix interpretation

These minimal algorithms for $F(m,k)$, $n = m + k - 1$, typically are written in matrix form as

$$\mathbf{z} = \mathbf{A}^{\mathsf{T}} \left((\mathbf{B}\mathbf{x}) \circ (\mathbf{C}\mathbf{f}) \right), \quad \mathbf{A} \in \mathbb{R}^{n \times m}, \quad \mathbf{B} \in \mathbb{R}^{n \times n}, \quad \mathbf{C} \in \mathbb{R}^{n \times k}.$$

Matrix interpretation

These minimal algorithms for $F(m,k)$, $n = m + k - 1$, typically are written in matrix form as

$$\mathbf{z} = \mathbf{A}^\mathsf{T} \left((\mathbf{B}\mathbf{x}) \circ (\mathbf{C}\mathbf{f}) \right), \quad \mathbf{A} \in \mathbb{R}^{n \times m}, \quad \mathbf{B} \in \mathbb{R}^{n \times n}, \quad \mathbf{C} \in \mathbb{R}^{n \times k}.$$

Here,

$$\mathbf{A}^\mathsf{T} = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} 1 & 0 & 0 \\ 1/2 & 1/2 & 1/2 \\ 1/2 & -1/2 & 1/2 \\ 0 & 0 & 1 \end{pmatrix}.$$

Matrix interpretation

These minimal algorithms for $F(m,k)$, $n = m + k - 1$, typically are written in matrix form as

$$\mathbf{z} = \mathbf{A}^\mathsf{T} \left((\mathbf{B}\mathbf{x}) \circ (\mathbf{C}\mathbf{f}) \right), \quad \mathbf{A} \in \mathbb{R}^{n \times m}, \quad \mathbf{B} \in \mathbb{R}^{n \times n}, \quad \mathbf{C} \in \mathbb{R}^{n \times k}.$$

Here,

$$\mathbf{A}^\mathsf{T} = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} 1 & 0 & 0 \\ 1/2 & 1/2 & 1/2 \\ 1/2 & -1/2 & 1/2 \\ 0 & 0 & 1 \end{pmatrix}.$$

This has similarities with the DFT/FFT approach:

transform to Winograd domain: $\mathbf{B}$ maps the signal $\mathbf{x} \in \mathbb{R}^n$ to its Winograd transform $\widehat{\mathbf{x}} \in \mathbb{R}^n$,

Matrix interpretation

These minimal algorithms for $F(m,k)$, $n = m + k - 1$, typically are written in matrix form as

$$\mathbf{z} = \mathbf{A}^\mathsf{T} \left((\mathbf{B}\mathbf{x}) \circ (\mathbf{C}\mathbf{f}) \right), \quad \mathbf{A} \in \mathbb{R}^{n \times m}, \quad \mathbf{B} \in \mathbb{R}^{n \times n}, \quad \mathbf{C} \in \mathbb{R}^{n \times k}.$$

Here,

$$\mathbf{A}^\mathsf{T} = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} 1 & 0 & 0 \\ 1/2 & 1/2 & 1/2 \\ 1/2 & -1/2 & 1/2 \\ 0 & 0 & 1 \end{pmatrix}.$$

This has similarities with the DFT/FFT approach:

transform to Winograd domain: $\mathbf{B}$ maps the signal $\mathbf{x} \in \mathbb{R}^n$ to its Winograd transform $\widehat{\mathbf{x}} \in \mathbb{R}^n$,

$\mathbf{C}$ maps the filter $\mathbf{f} \in \mathbb{R}^k$ to its Winograd transform $\widehat{\mathbf{f}} \in \mathbb{R}^n$;

Matrix interpretation

These minimal algorithms for $F(m,k)$, $n = m + k - 1$, typically are written in matrix form as

$$\mathbf{z} = \mathbf{A}^\mathsf{T} \left((\mathbf{B}\mathbf{x}) \circ (\mathbf{C}\mathbf{f}) \right), \quad \mathbf{A} \in \mathbb{R}^{n \times m}, \quad \mathbf{B} \in \mathbb{R}^{n \times n}, \quad \mathbf{C} \in \mathbb{R}^{n \times k}.$$

Here,

$$\mathbf{A}^\mathsf{T} = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} 1 & 0 & 0 \\ 1/2 & 1/2 & 1/2 \\ 1/2 & -1/2 & 1/2 \\ 0 & 0 & 1 \end{pmatrix}.$$

This has similarities with the DFT/FFT approach:

transform to Winograd domain: $\mathbf{B}$ maps the signal $\mathbf{x} \in \mathbb{R}^n$ to its Winograd transform $\widehat{\mathbf{x}} \in \mathbb{R}^n$,

$\mathbf{C}$ maps the filter $\mathbf{f} \in \mathbb{R}^k$ to its Winograd transform $\widehat{\mathbf{f}} \in \mathbb{R}^n$;

multiply componentwise: $\widehat{\mathbf{z}} = \widehat{\mathbf{x}} \circ \widehat{\mathbf{f}}$ is computed in Winograd domain $\mathbb{R}^n$;

Matrix interpretation

These minimal algorithms for $F(m,k)$, $n = m + k - 1$, typically are written in matrix form as

$$\mathbf{z} = \mathbf{A}^\mathsf{T} \left((\mathbf{B}\mathbf{x}) \circ (\mathbf{C}\mathbf{f}) \right), \quad \mathbf{A} \in \mathbb{R}^{n \times m}, \quad \mathbf{B} \in \mathbb{R}^{n \times n}, \quad \mathbf{C} \in \mathbb{R}^{n \times k}.$$

Here,

$$\mathbf{A}^\mathsf{T} = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} 1 & 0 & 0 \\ 1/2 & 1/2 & 1/2 \\ 1/2 & -1/2 & 1/2 \\ 0 & 0 & 1 \end{pmatrix}.$$

This has similarities with the DFT/FFT approach:

transform to Winograd domain: $\mathbf{B}$ maps the signal $\mathbf{x} \in \mathbb{R}^n$ to its Winograd transform $\widehat{\mathbf{x}} \in \mathbb{R}^n$,
$\mathbf{C}$ maps the filter $\mathbf{f} \in \mathbb{R}^k$ to its Winograd transform $\widehat{\mathbf{f}} \in \mathbb{R}^n$;

multiply componentwise: $\widehat{\mathbf{z}} = \widehat{\mathbf{x}} \circ \widehat{\mathbf{f}}$ is computed in Winograd domain $\mathbb{R}^n$;

transform back: $\mathbf{A}^\mathsf{T}$ maps the result from Winograd domain $\mathbb{R}^n$ to output domain $\mathbb{R}^m$.

Matrix interpretation

These minimal algorithms for $F(m,k)$, $n = m + k - 1$, typically are written in matrix form as

$$\mathbf{z} = \mathbf{A}^\mathsf{T} \left((\mathbf{B}\mathbf{x}) \circ (\mathbf{C}\mathbf{f}) \right), \quad \mathbf{A} \in \mathbb{R}^{n \times m}, \quad \mathbf{B} \in \mathbb{R}^{n \times n}, \quad \mathbf{C} \in \mathbb{R}^{n \times k}.$$

Here, flipping filter $\rightsquigarrow$ flipping $\mathbf{C}$ initially is faster

$$\mathbf{A}^\mathsf{T} = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix}, \quad \widetilde{\mathbf{C}} = \begin{pmatrix} 0 & 0 & 1 \\ 1/2 & 1/2 & 1/2 \\ 1/2 & -1/2 & 1/2 \\ 1 & 0 & 0 \end{pmatrix}.$$

This has similarities with the DFT/FFT approach:

transform to Winograd domain: $\mathbf{B}$ maps the signal $\mathbf{x} \in \mathbb{R}^n$ to its Winograd transform $\widehat{\mathbf{x}} \in \mathbb{R}^n$,
$\mathbf{C}$ maps the filter $\mathbf{f} \in \mathbb{R}^k$ to its Winograd transform $\widehat{\mathbf{f}} \in \mathbb{R}^n$;

multiply componentwise: $\widehat{\mathbf{z}} = \widehat{\mathbf{x}} \circ \widehat{\mathbf{f}}$ is computed in Winograd domain $\mathbb{R}^n$;

transform back: $\mathbf{A}^\mathsf{T}$ maps the result from Winograd domain $\mathbb{R}^n$ to output domain $\mathbb{R}^m$.

Winograd's minimal algorithm for $F(2 \times 2, 3 \times 3)$

Minimal FIR algorithms can be nested to apply to higher dimensions, e.g., in 2D,

$$\mathbf{Z} = \mathbf{A}_1^\mathsf{T} \left[\mathbf{B}_1 \mathbf{X} \mathbf{B}_2^\mathsf{T} \circ \mathbf{C}_1 \mathbf{F} \mathbf{C}_2^\mathsf{T} \right] \mathbf{A}_2,$$

the complexity of $F(m_1 \times m_2, k_1 \times k_2)$ is given by $(m_1 + k_1 - 1)(m_2 + k_2 - 1)$.

Winograd's minimal algorithm for $F(2 \times 2, 3 \times 3)$

Minimal FIR algorithms can be nested to apply to higher dimensions, e.g., in 2D,

$$\mathbf{Z} = \mathbf{A}_1^\mathsf{T} \left(\mathbf{B}_1 \mathbf{X} \mathbf{B}_2^\mathsf{T} \circ \mathbf{C}_1 \mathbf{F} \mathbf{C}_2^\mathsf{T} \right) \mathbf{A}_2,$$

the complexity of $F(m_1 \times m_2, k_1 \times k_2)$ is given by $(m_1 + k_1 - 1)(m_2 + k_2 - 1)$.

Thus, for $F(2 \times 2, 3 \times 3)$ (valid 2D cross-correlation of 4×4 patches with a 3×3 kernel), the improvement on the number of multiplications is

$$\frac{6 \times 6}{4 \times 4} = \frac{9}{4} = 2.25.$$

Winograd's minimal algorithm for $F(2 \times 2, 3 \times 3)$

Minimal FIR algorithms can be nested to apply to higher dimensions, e.g., in 2D,

$$\mathbf{Z} = \mathbf{A}_1^\mathsf{T} \left(\mathbf{B}_1 \mathbf{X} \mathbf{B}_2^\mathsf{T} \circ \mathbf{C}_1 \mathbf{F} \mathbf{C}_2^\mathsf{T} \right) \mathbf{A}_2,$$

the complexity of $F(m_1 \times m_2, k_1 \times k_2)$ is given by $(m_1 + k_1 - 1)(m_2 + k_2 - 1)$.

Thus, for $F(2 \times 2, 3 \times 3)$ (valid 2D cross-correlation of 4×4 patches with a 3×3 kernel), the improvement on the number of multiplications is

$$\frac{6 \times 6}{4 \times 4} = \frac{9}{4} = 2.25.$$

Questions:

- Does reducing the number of multiplications make the algorithm faster?
  Let us consider the 4-tensor case (NCHW).
- How does Winograd construct minimal algorithms? $\rightsquigarrow$ Lecture Notes

Implementation of Winograd's convolution

In Lavin and Gray [7] the implementation of Winograd's minimal algorithm for $F(m \times m, r \times r)$ is dicussed, first for $m = 2, r = 3$ (feedforward), then $m = 3, r = 2$ (backpropagation).

Implementation of Winograd's convolution

In Lavin and Gray [7] the implementation of Winograd's minimal algorithm for $F(m \times m, r \times r)$ is dicussed, first for $m = 2, r = 3$ (feedforward), then $m = 3, r = 2$ (backpropagation).

The authors divide the $h \times w$ images per channel in tiles of size $(m + r - 1) \times (m + r - 1)$ with overlap $r - 1$, yielding $P = \lceil h/m \rceil \cdot \lceil w/m \rceil$ tiles per channel.

Implementation of Winograd's convolution

In Lavin and Gray [7] the implementation of Winograd's minimal algorithm for $F(m \times m, r \times r)$ is dicussed, first for $m = 2, r = 3$ (feedforward), then $m = 3, r = 2$ (backpropagation).

The authors divide the $h \times w$ images per channel in tiles of size $(m + r - 1) \times (m + r - 1)$ with overlap $r - 1$, yielding $P = \lceil h/m \rceil \cdot \lceil w/m \rceil$ tiles per channel.

Let $\mathbf{X}(i, k, \widetilde{x}, \widetilde{y}) \in \mathbb{R}^{(m+r-1)\times(m+r-1)}$ denote the tile with coordinates $(\widetilde{x}, \widetilde{y})$ of image i for channel k. We have to compute for every image i and every filter $\mathbf{F}(j, k) \in \mathbb{R}^{r\times r}$ the sum

$$\mathbf{Z}(i, j, \widetilde{x}, \widetilde{y}) = \sum_{k=1}^{c} \mathbf{X}(i, k, \widetilde{x}, \widetilde{y}) \star \mathbf{F}(j, k) = \mathbf{A}^{\mathsf{T}} \left(\sum_{k=1}^{c} \left(\mathbf{B}\mathbf{X}(i, k, \widetilde{x}, \widetilde{y})\mathbf{B}^{\mathsf{T}} \right) \circ \left(\mathbf{C}\mathbf{F}(j, k)\mathbf{C}^{\mathsf{T}} \right) \right) \mathbf{A}.$$

Implementation of Winograd's convolution

In Lavin and Gray [7] the implementation of Winograd's minimal algorithm for $F(m \times m, r \times r)$ is dicussed, first for $m = 2, r = 3$ (feedforward), then $m = 3, r = 2$ (backpropagation).

The authors divide the $h \times w$ images per channel in tiles of size $(m + r - 1) \times (m + r - 1)$ with overlap $r - 1$, yielding $P = \lceil h/m \rceil \cdot \lceil w/m \rceil$ tiles per channel.

Let $\mathbf{X}(i, k, \widetilde{x}, \widetilde{y}) \in \mathbb{R}^{(m+r-1)\times(m+r-1)}$ denote the tile with coordinates $(\widetilde{x}, \widetilde{y})$ of image i for channel k. We have to compute for every image i and every filter $\mathbf{F}(j, k) \in \mathbb{R}^{r\times r}$ the sum

$$\mathbf{Z}(i, j, \widetilde{x}, \widetilde{y}) = \sum_{k=1}^{c} \mathbf{X}(i, k, \widetilde{x}, \widetilde{y}) \star \mathbf{F}(j, k) = \mathbf{A}^\mathsf{T} \left(\sum_{k=1}^{c} \left(\mathbf{B}\mathbf{X}(i, k, \widetilde{x}, \widetilde{y})\mathbf{B}^\mathsf{T} \right) \circ \left(\mathbf{C}\mathbf{F}(j, k)\mathbf{C}^\mathsf{T} \right) \right) \mathbf{A}.$$

- Why should this speed up computation?
- How can we implement this idea in Python? $\rightsquigarrow$ Lecture Notes

Improvement over direct convolution

Image size $C \times H \times W$, filter bank size $M \times C \times 3 \times 3$. We consider $F(2 \times 2, 3 \times 3)$, $(H/2)(W/2)$ tiles.

Improvement over direct convolution

Image size $C \times H \times W$, filter bank size $M \times C \times 3 \times 3$. We consider $F(2 \times 2, 3 \times 3)$, $(H/2)(W/2)$ tiles.

Direct convolution: $\approx 9MCHW = O(MCHW)$ operations

Improvement over direct convolution

Image size $C \times H \times W$, filter bank size $M \times C \times 3 \times 3$. We consider $F(2 \times 2, 3 \times 3)$, $(H/2)(W/2)$ tiles.

Direct convolution: $\approx 9MCHW = O(MCHW)$ operations

Image transform: $(4 \cdot 4 + 4 \cdot 4)C(H/2)(W/2) = 8CHW = O(CHW)$ additions

Improvement over direct convolution

Image size $C \times H \times W$, filter bank size $M \times C \times 3 \times 3$. We consider $F(2 \times 2, 3 \times 3)$, $(H/2)(W/2)$ tiles.

Direct convolution: $\approx 9MCHW = O(MCHW)$ operations

Image transform: $(4 \cdot 4 + 4 \cdot 4)C(H/2)(W/2) = 8CHW = O(CHW)$ additions

Filter transform: $28MC = O(MC)$ floating point operations

Improvement over direct convolution

Image size $C \times H \times W$, filter bank size $M \times C \times 3 \times 3$. We consider $F(2 \times 2, 3 \times 3)$, $(H/2)(W/2)$ tiles.

Direct convolution: $\approx 9MCHW = O(MCHW)$ operations

Image transform: $(4 \cdot 4 + 4 \cdot 4)C(H/2)(W/2) = 8CHW = O(CHW)$ additions

Filter transform: $28MC = O(MC)$ floating point operations

Winograd product: $16MC(H/2)(W/2) = 4MCHW = O(MCHW)$ operations

Improvement over direct convolution

Image size $C \times H \times W$, filter bank size $M \times C \times 3 \times 3$. We consider $F(2 \times 2, 3 \times 3)$, $(H/2)(W/2)$ tiles.

Direct convolution: $\approx 9MCHW = O(MCHW)$ operations

Image transform: $(4 \cdot 4 + 4 \cdot 4)C(H/2)(W/2) = 8CHW = O(CHW)$ additions

Filter transform: $28MC = O(MC)$ floating point operations

Winograd product: $16MC(H/2)(W/2) = 4MCHW = O(MCHW)$ operations

Backtransform: $24M(H/2)(W/2) = 6MHW = O(MHW)$ additions

Improvement over direct convolution

Image size $C \times H \times W$, filter bank size $M \times C \times 3 \times 3$. We consider $F(2 \times 2, 3 \times 3)$, $(H/2)(W/2)$ tiles.

Direct convolution: $\approx 9MCHW = O(MCHW)$ operations

Image transform: $(4 \cdot 4 + 4 \cdot 4)C(H/2)(W/2) = 8CHW = O(CHW)$ additions

Filter transform: $28MC = O(MC)$ floating point operations

Winograd product: $16MC(H/2)(W/2) = 4MCHW = O(MCHW)$ operations

Backtransform: $24M(H/2)(W/2) = 6MHW = O(MHW)$ additions

If $M \gg 8$, $HW \gg 28$, $C \gg 6$, we can neglect the operations for the tranforms, and Winograd convolution is dominated by $4MCHW$ operations, giving an increase of 2.25 in speed.

Improvement over direct convolution

Image size $C \times H \times W$, filter bank size $M \times C \times 3 \times 3$. We consider $F(2 \times 2, 3 \times 3)$, $(H/2)(W/2)$ tiles.

Direct convolution: $\approx 9MCHW = O(MCHW)$ operations

Image transform: $(4 \cdot 4 + 4 \cdot 4)C(H/2)(W/2) = 8CHW = O(CHW)$ additions

Filter transform: $28MC = O(MC)$ floating point operations

Winograd product: $16MC(H/2)(W/2) = 4MCHW = O(MCHW)$ operations

Backtransform: $24M(H/2)(W/2) = 6MHW = O(MHW)$ additions

If $M \gg 8$, $HW \gg 28$, $C \gg 6$, we can neglect the operations for the tranforms, and Winograd convolution is dominated by $4MCHW$ operations, giving an increase of 2.25 in speed.

Due to missing data locality the improvement will be less than 2.25.

References I

[1] Dan Ciresan et al. “Flexible, High Performance Convolutional Neural Networks for Image Classification”. In: *Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence*. Vol. 2. 2011, pp. 1237–1242.

[2] James W. Cooley and John W. Tukey. “An algorithm for the machine calculation of complex Fourier series”. In: *Math. Comp.* 19 (1965), pp. 297–301. ISSN: 0025-5718. DOI: 10.2307/2003354.

[3] Kunihiko Fukushima. “Neocognitron: A Self-organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position”. In: *Biological Cybernetics* 36.4 (1980), pp. 193–202. DOI: 10.1007/BF00344251. URL: https://www.cs.princeton.edu/courses/archive/spr08/cos598B/Readings/Fukushima1980.pdf.

[4] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2016. DOI: 10.1109/CVPR.2016.90.

References II

[5] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems 25*. Ed. by F. Pereira et al. Curran Associates, Inc., 2012, pp. 1097–1105. URL: http://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf.

[6] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. “ImageNet classification with deep convolutional neural networks”. In: *Communications of the ACM* 60.6 (2017), pp. 84–90. ISSN: 0001-0782. DOI: 10.1145/3065386.

[7] Andrew Lavin and Scott Gray. “Fast Algorithms for Convolutional Neural Networks”. In: *arXiv e-prints*, arXiv:1509.09308 (Sept. 2015). arXiv: 1509.09308 [cs.NE].

[8] Y. LeCun et al. “Backpropagation Applied to Handwritten Zip Code Recognition”. In: *Neural Computation* 1 (1989), pp. 541–551. URL: http://yann.lecun.com/exdb/publis/pdf/lecun-89e.pdf.

References III

[9] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (Nov. 1998), pp. 2278–2324. DOI: 10.1109/5.726791. URL: http://yann.lecun.com/exdb/mnist/.

[10] J Weng, N Ahuja, and TS Huang. “Learning recognition and segmentation of 3-D objects from 2-D images”. In: *Proc. 4th International Conf. Computer Vision*. 1993, pp. 121–128.

[11] Shmuel Winograd. *Arithmetic complexity of computations*. Vol. 33. CBMS-NSF Regional Conference Series in Applied Mathematics. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, Pa., 1980, pp. iii+93. ISBN: 0-89871-163-0.

[12] Kouichi Yamaguchi et al. “A Neural Network for Speaker-Independent Isolated Word Recognition”. In: *First International Conference on Spoken Language Processing (ICSLP 90)*. Kobe, Japan, Nov. 1990. URL: https://www.isca-speech.org/archive/icslp_1990/i90_1077.html.